

ANOTAÇÕES KUBERNETS

Documentação instalação

<https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/install-kubeadm/>

Para criar um cluster kubernetes, é necessário realizar algumas configurações em todos os NODES (master e workers)!

1) Desabilitar a memória swap

```
$ swapoff -a
```

```
$ vi /etc/fstab (comente ou exclua a linha que faça referencia a swap, caso tenha).
```

2) Desabilite o SELinux

```
$ setenforce 0
```

```
$ sed -i 's/^SELINUX=enforcing$/SELINUX=disabled/' /etc/sysconfig/selinux
```

```
$ reboot
```

3) Desligue o firewalld (ou habilite as portas necessárias)

```
$ systemctl stop firewalld
```

```
$ systemctl disable firewalld
```

Ou

```
$ firewall-cmd --permanent --add-port={6443,2379,2380,10250,10251,10252,10259,179}/tcp
```

```
$ firewall-cmd --permanent --add-port=4789/udp
```

```
$ firewall-cmd --reload
```

4) Para o cluster kubernetes, devemos adicionar os módulos de kernel overlay e br_netfilter em todos os nós. Crie um arquivo e adicione o seguinte conteúdo a ele,

```
$ tee /etc/modules-load.d/containerd.conf <<EOF
```

```
overlay
br_netfilter
EOF
```

```
$ sudo modprobe overlay
$ sudo modprobe br_netfilter
```

Em seguida, adicione o seguinte conteúdo no arquivo `/etc/sysctl.d/k8s.conf`

```
$ vi /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-ip6tables = 1
```

Adicione os parâmetros

```
$ sysctl --system
```

5) Instale o container runtime (containerd)

É necessário adicionar o repositório para ele conseguir achar o pacote

```
$ dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
```

```
$ yum install containerd.io -y
```

6) Configure o containerd para que ele use o `systemd` e execute os seguintes comandos em cada nó.

```
$ containerd config default | tee /etc/containerd/config.toml >/dev/null 2>&1
$ sed -i 's/SystemdCgroup \= false/SystemdCgroup \= true/g' /etc/containerd/config.toml
```

```
$ sudo systemctl restart containerd
```

```
$ sudo systemctl enable containerd
```

7) Instale as seguintes ferramentas do kubernetes:

kubeadm: the command to bootstrap the cluster.

kubelet: the component that runs on all of the machines in your cluster and does things like starting pods and containers.

kubectrl: the command line util to talk to your cluster.

Adicione o repo

```
$ cat <<EOF | tee /etc/yum.repos.d/kubernetes.repo
```

```
[kubernetes]
```

```
name=Kubernetes
```

```
baseurl=https://pkgs.k8s.io/core:/stable:/v1.28/rpm/
```

```
enabled=1
```

```
gpgcheck=1
```

```
gpgkey=https://pkgs.k8s.io/core:/stable:/v1.28/rpm/repodata/repomd.xml.key
```

```
exclude=kubelet kubeadm kubectl cri-tools kubernetes-cni
```

```
EOF
```

```
$ yum install -y kubelet kubeadm kubectl --disableexcludes=kubernetes
```

```
$ systemctl enable --now kubelet
```

7) export KUBECONFIG=/etc/kubernetes/admin.conf (coloque no /etc/profile - somente no master!!!!)

Faça isso nos workers

You most probably followed the these 4 lines:

```
$ mkdir -p $HOME/.kube
```

```
$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

```
$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Criando o cluster com o kubeadm - somente no master!

```
$ kubeadm init --apiserver-advertise-address 172.89.0.11 --pod-network-cidr 10.244.0.0/16
```

--apiserver-advertise-address

Esse parâmetro é muito importante pois ele define a interface de rede padrão que o cluster irá trabalhar. Caso você não a use, ele vai pegar a interface padrão, que pode não contemplar a rede em que os workers operam.

--pod-network-cidr

Aqui é definido a cidr dos pods, que no kubernetes é chamado CNI (Container Network Interface), e existem vários plugins que oferecem uma gama funcionalidades, além da rede para os pods. A que irei usar é o Flannel (<https://github.com/flannel-io/flannel>)

Lista os nodes (master, workers...)

```
$ kubectl get node
```

```
$ kubectl get node -A
```

| -> Lista os nodes em todas as namespaces

```
$ kubeadm token create --print-join-command
```

Esse comando cria um token de autenticação para os workers ingressarem no cluster

```
$ kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
```

Criando a CNI (Flannel) Isso é necessário para que o coredns consiga entregar endereços IP's para os nodes dentro do cluster

Obtendo mais informações do cluster (explorando o get)

Mostra quais os recursos de API disponíveis no cluster kubernetes

```
$ kubectl api-resources
```

```
$ kubectl get pod -A
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-flannel	kube-flannel-ds-jfwbs	1/1	Running	0	121m
kube-flannel	kube-flannel-ds-lv2b7	1/1	Running	1 (75m ago)	121m
kube-flannel	kube-flannel-ds-thxnp	1/1	Running	1 (75m ago)	121m
kube-system	coredns-5dd5756b68-bz4qf	1/1	Running	1 (75m ago)	24h
kube-system	coredns-5dd5756b68-w6sp9	1/1	Running	1 (75m ago)	24h
kube-system	etcd-master-1	1/1	Running	53 (75m ago)	24h
kube-system	kube-apiserver-master-1	1/1	Running	53 (75m ago)	24h
kube-system	kube-controller-manager-master-1	1/1	Running	57 (75m ago)	24h
kube-system	kube-proxy-pc97p	1/1	Running	12 (75m ago)	24h
kube-system	kube-proxy-pt4vn	1/1	Running	1 (75m ago)	129m
kube-system	kube-proxy-qsqtv	1/1	Running	0	129m
kube-system	kube-scheduler-master-1	1/1	Running	54 (75m ago)	24h

Como seria possível listar apenas um recurso desses acima? por exemplo, o etcd

```
$ kubectl get pod -n kube-system etcd-master-1
```

NAME	READY	STATUS	RESTARTS	AGE
etcd-master-1	1/1	Running	53 (77m ago)	24h

É possível exportar a saída para o formato JSOM ou YAML utilizando a opção "-o"

```
$ kubectl get pod -n kube-system etcd-master-1 -o json
```

```
$ kubectl get pod -n kube-system etcd-master-1 -o yaml
```

Podemos obter o endereço IP de um pod através do comando abaixo

```
$ kubectl get pod -A -owide
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
kube-flannel	kube-flannel-ds-jfwbs	1/1	Running	0	139m	10.0.2.15	
worker-1	<none>						
kube-flannel	kube-flannel-ds-lv2b7	1/1	Running	1 (93m ago)	139m	10.0.2.15	
master-1	<none>						
kube-flannel	kube-flannel-ds-thxnp	1/1	Running	1 (93m ago)	139m	10.0.2.15	
worker-2	<none>						
kube-system	coredns-5dd5756b68-bz4qf	1/1	Running	1 (93m ago)	24h		
10.244.2.5	worker-2	<none>					
kube-system	coredns-5dd5756b68-w6sp9	1/1	Running	1 (93m ago)	24h		
10.244.2.4	worker-2	<none>					
kube-system	etcd-master-1	1/1	Running	53 (93m ago)	24h	10.0.2.15	
master-1	<none>						
kube-system	kube-apiserver-master-1	1/1	Running	53 (93m ago)	24h	10.0.2.15	
master-1	<none>						
kube-system	kube-controller-manager-master-1	1/1	Running	57 (93m ago)	24h		
10.0.2.15	master-1	<none>					
kube-system	kube-proxy-pc97p	1/1	Running	12 (93m ago)	24h	10.0.2.15	
master-1	<none>						
kube-system	kube-proxy-pt4vn	1/1	Running	1 (93m ago)	147m	10.0.2.15	
worker-2	<none>						
kube-system	kube-proxy-qsqtv	1/1	Running	0	147m	10.0.2.15	worker-
1	<none>						
kube-system	kube-scheduler-master-1	1/1	Running	54 (93m ago)	24h	10.0.2.15	
master-1	<none>						

Da para filtrar por labels, cada componente possui um label.

```
$ kubectl get pod -A --show-labels
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE	LABELS
kube-flannel	kube-flannel-ds-jfwbs	1/1	Running	0	141m	app=flannel,controller-revision-hash=6ddfb4b8,k8s-app=flannel,pod-template-generation=1,tier=node

```

kube-flannel kube-flannel-ds-lv2b7      1/1   Running  1 (94m ago)  141m
app=flannel,controller-revision-hash=6ddfb4b8,k8s-app=flannel,pod-template-
generation=1,tier=node

kube-flannel kube-flannel-ds-thxnp      1/1   Running  1 (94m ago)  141m
app=flannel,controller-revision-hash=6ddfb4b8,k8s-app=flannel,pod-template-
generation=1,tier=node

kube-system coredns-5dd5756b68-bz4qf    1/1   Running  1 (94m ago)  24h   k8s-
app=kube-dns,pod-template-hash=5dd5756b68

kube-system coredns-5dd5756b68-w6sp9    1/1   Running  1 (94m ago)  24h   k8s-
app=kube-dns,pod-template-hash=5dd5756b68

kube-system etcd-master-1                1/1   Running  53 (94m ago)  24h
component=etcd,tier=control-plane

kube-system kube-apiserver-master-1      1/1   Running  53 (94m ago)  24h
component=kube-apiserver,tier=control-plane

kube-system kube-controller-manager-master-1 1/1   Running  57 (94m ago)  24h
component=kube-controller-manager,tier=control-plane

kube-system kube-proxy-pc97p             1/1   Running  12 (94m ago)  24h   controller-
revision-hash=cbffbc7d4,k8s-app=kube-proxy,pod-template-generation=1

kube-system kube-proxy-pt4vn             1/1   Running  1 (94m ago)  148m   controller-
revision-hash=cbffbc7d4,k8s-app=kube-proxy,pod-template-generation=1

kube-system kube-proxy-qsqtv             1/1   Running  0           148m   controller-revision-
hash=cbffbc7d4,k8s-app=kube-proxy,pod-template-generation=1

kube-system kube-scheduler-master-1      1/1   Running  54 (94m ago)  24h
component=kube-scheduler,tier=control-plane

```

Listando somente o pod dns através do label

```
$ kubectl get pod -A -l k8s-app=kube-dns
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	coredns-5dd5756b68-bz4qf	1/1	Running	1 (95m ago)	24h
kube-system	coredns-5dd5756b68-w6sp9	1/1	Running	1 (95m ago)	24h

Configurando o etcdctl

O etcdctl é uma ferramenta a parte do kubernetes, usada normalmente para interagir o com o banco de dados do kubernetes, etcd.

E porque é interessante interagir com o etcd?

Por que podemos fazer backup e snapshot do etcd, lembrando que esse recurso é totalmente crítico dentro do cluster, se alguma coisa der errado no etcd, todo o cluster será impactado.

Para encontrar o pacote do etcdctl, baixei o repositório abaixo

```
$ yum install -y centos-release-openstack-caracal.noarch
```

Depois, instalei o pacote do etcdctl

```
$ yum install -y etcd
```

Para usar, a própria documentação fala que devemos utilizar a variável: ETCDCTL_API=3

Além disso, é necessário passar diversos outros parametros para autenticar.

```
--cacert
```

```
--cert
```

```
--key
```

E onde buscamos essas informações, referente ao etcd?

No diretório: /etc/kubernetes/pki/etcd/

```
[root@master-1 ~]# ls -l /etc/kubernetes/pki/etcd/
```

```
total 32
```

```
-rw-r--r-- 1 root root 1094 Sep 11 00:11 ca.crt <-
```

```
-rw----- 1 root root 1679 Sep 11 00:11 ca.key
```

```
-rw-r--r-- 1 root root 1159 Sep 11 00:11 healthcheck-client.crt
```

```
-rw----- 1 root root 1679 Sep 11 00:11 healthcheck-client.key
```

```
-rw-r--r-- 1 root root 1196 Sep 11 00:11 peer.crt
```

```
-rw----- 1 root root 1675 Sep 11 00:11 peer.key
```

```
-rw-r--r-- 1 root root 1196 Sep 11 00:11 server.crt <-
```

```
-rw----- 1 root root 1675 Sep 11 00:11 server.key <-
```

Como fica o comando:

```
$ ETCDCTL_API=3 etcdctl --cacert /etc/kubernetes/pki/etcd/ca.crt --cert  
/etc/kubernetes/pki/etcd/server.crt --key /etc/kubernetes/pki/etcd/server.key member list --  
write-out="table"
```

```
+-----+-----+-----+-----+-----+-----+  
|  ID   | STATUS | NAME   | PEER ADDRS | CLIENT ADDRS | IS LEARNER |  
+-----+-----+-----+-----+-----+-----+  
| d06fe5b202c2f3b3 | started | master-1 | https://172.89.0.11:2380 |  
https://172.89.0.11:2379 | false |  
+-----+-----+-----+-----+-----+-----+
```

o parametro `--write-out="table"` é somente para visualizar em modo de tabela.

É possível criar um script para otimizar um pouco isso:

```
cat <<EOF > etcdctl.env  
  
export ETCDCTL_CACERT=/etc/kubernetes/pki/etcd/ca.crt  
export ETCDCTL_CERT=/etc/kubernetes/pki/etcd/server.crt  
export ETCDCTL_KEY=/etc/kubernetes/pki/etcd/server.key  
export ETCDCTL_API=3  
  
EOF
```

```
$ source etcdctl.env
```

```
$ etcdctl member list --write-out="table"
```

```
+-----+-----+-----+-----+-----+-----+  
|  ID   | STATUS | NAME   | PEER ADDRS | CLIENT ADDRS | IS LEARNER |  
+-----+-----+-----+-----+-----+-----+  
| d06fe5b202c2f3b3 | started | master-1 | https://172.89.0.11:2380 |  
https://172.89.0.11:2379 | false |  
+-----+-----+-----+-----+-----+-----+
```

Fazendo restore da base etcd

Geralmente é realizado no `/var/lib/etcd` e basicamente é criado um arquivo.

```
$ etcdctl snapshot save snapshot.db
```

Para restaurar:

```
$ etcdctl snapshot restore snapshot.db
```

Usando arquivos YAML com Kubernetes

O arquivo usado na aula é esse abaixo:

pod.yaml

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: nginx
```

```
spec:
```

```
  containers:
```

```
    - name: nginx
```

```
      image: nginx:1.14.2
```

```
      ports:
```

```
        - containerPort: 80
```

Ele basicamente cria um pod que roda o nginx e especifica a porta 80.

Com o arquivo criado, execute:

```
$ kubectl apply -f pod.yaml
```

Comando para criar um Pod utilizando um arquivo yaml

```
# Obtendo YAML de objetos + krew
```

Para obter o manifesto yaml de um POD, utilizamos a opção "-o", como já visto antes.

```
$ kubectl get pod nginx -oyaml
```

E é possível coletar o yaml de literalmente qualquer objeto, não apenas um pod, mas também serviços, config maps e etc.

```
$ kubectl get service kubernetes -oyaml
```

```
$ kubectl get cm kube-root-ca.crt -oyaml
```

É possível reutilizar essas configurações para criar outro objeto, como um pod por exemplo.

Entretanto, essas configurações possuem metadados que não são interessantes na criação de outro recurso, a exemplo de timestamp de criação entre outras coisas que podemos considerar como "lixo".

Ai que entra o "krew", que é uma extensão do kubectl que nos permite instalar diversos plugins.

Para instalar, siga a documentação abaixo

<https://krew.sigs.k8s.io/docs/user-guide/setup/install/#windows>

Um dos plugins que irei testar é o "neat", esse plugin consegue retirar os metadados desnecessários dos arquivos de configuração dos objetos de forma automática.

Para instalar:

```
$ kubectl krew install neat
```

O neat espera os dados no stdin, ou seja, para utilizá-lo é necessário redirecionar a saída.

```
$ kubectl get pod nginx -oyaml | kubectl neat
```

```
$ kubectl get pod nginx -oyaml | kubectl neat > pod.yaml
```

Usando namespaces na prática

Uma namespace basicamente é um espaço lógico isolado, dentro do cluster kubernetes, ajudando bastante na questão de ornalização e na granularização do acesso.

```
$ kubectl get namespace
```

```
$ kubectl get ns
```

NAME	STATUS	AGE
default	Active	10d
kube-flannel	Active	10d
kube-node-lease	Active	10d
kube-public	Active	10d
kube-system	Active	10d

Um ponto importante pare se notar é que por padrão, todo comando kubectl listará a namespace "default", caso você não especifique uma namespace.

Para especificar uma namespace:

```
$ kubectl get pod --namespace kube-system
```

```
$ kubectl get pod --n kube-system
```

É possível mudar a namespace padrão (porém isso não é recomendado!)

```
$ kubectl config set-context --current --namespace kube-system
```

Formas de criar uma nova namespace

1) através do kubectl create

```
$ kubectl create namespace teste
```

```
$ kubectl create ns teste
```

```
namespace/teste created
```

Para deletar:

```
$ kubectl delete ns teste
```

2) Através de um arquivo YAML

Para gerar o arquivo yaml:

```
$ kubectl create ns teste --dry-run
```

A opção "--dry-run" não cria o objeto de fato e com isso podemos utilizar a opção -o yaml junto com o neat!

```
$ kubectl create ns teste --dry-run -oyaml | kubectl neat > /tmp/Pod.yaml
```

Depois podemos aplicar?

```
$ kubectl apply -f /tmp/Pod.yaml
```

Consultando logs do pod + troubleshooting envolvendo cluster criado com Vagrant

Para consultar logs em um pod:

```
$ kubectl logs -n teste nginx-teste
```

Error from server (NotFound): the server could not find the requested resource (pods/log nginx-teste)

Porém é comum aparecer esse erro caso você tenha criado o seu cluster via Vagrant, vamos entender o por que:

Primero, vamos debugar a solicitação API que é realizada nessa requisição de logs

```
$ kubectl logs -n teste nginx-teste -v9
```

A requisição nos retorna um erro 404 de notfound

```
"metadata": {},  
"status": "Failure",  
"message": "the server could not find the requested resource ( pods/log nginx-teste)",  
"reason": "NotFound",  
"details": {  
  "name": "nginx-teste",  
  "kind": "pods/log"  
},  
"code": 404  
}}
```

O motivo disso, é por que se faz um get no endereço IP e porta da worker em que esse pod está sendo executado

```
GET https://172.89.0.11:6443/api/v1/namespaces/teste/pods/nginx-teste/log?container=nginx
```

Entretanto, embora esteja correto o IP, o kubectl está enxergando apenas a interface de NAT, criada pelo Vagrant, conforme é possível visualizar abaixo

```
$ kubectl get po -n teste nginx-teste -owide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE
nginx-teste	1/1	Running	1 (16m ago)	45h	10.244.1.5	worker-1	<none>	<none>

Para contornar isso, existem algumas formas, essa que tentei, não funcionou comigo, porém, vou deixar anotado

1)

Devemos alterar o arquivo de configuração do serviço kubelet:
/usr/lib/systemd/system/kubelet.service.d

Na chamada do serviço do kubelet, tem uma variável que podemos usar para setar qual endereço IP o kubelet vai enxergar no cluster

```
ExecStart=/usr/bin/kubelet $KUBELET_KUBECONFIG_ARGS $KUBELET_CONFIG_ARGS  
$KUBELET_KUBEADM_ARGS $KUBELET_EXTRA_ARGS
```

Basta configurar no arquivo de configuração

```
Environment="KUBELET_EXTRA_ARGS=--node-ip=172.89.0.21"
```

Depois, reinicie o serviço

```
$ systemctl daemon-reload
```

```
$ systemctl restart kubelet
```

O que funcionou foi:

2)

Simplesmente colocar o endereço IP correto no /etc/hosts

```
[root@worker-1 ~]# cat /etc/hosts | grep -v ^# | grep worker-1  
172.89.0.21 worker-1
```

```
[root@worker-2 ~]# cat /etc/hosts | grep -v ^# | grep worker-2  
172.89.0.22 worker-2
```

Depois, reinicie o serviço

```
$ systemctl daemon-reload
```

```
$ systemctl restart kubelet
```

```
$ kubectl get node -owide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE
KERNEL-VERSION			CONTAINER-RUNTIME				
master-1	Ready	control-plane	12d	v1.28.13	10.0.2.15	<none>	Rocky Linux 9.4
(Blue Onyx) 5.14.0-427.13.1.el9_4.x86_64			containerd://1.7.21				
worker-1	Ready	<none>	11d	v1.28.13	172.89.0.21	<none>	Rocky Linux 9.4 (Blue
Onyx) 5.14.0-427.13.1.el9_4.x86_64			containerd://1.7.21				
worker-2	Ready	<none>	11d	v1.28.13	172.89.0.22	<none>	Rocky Linux 9.4 (Blue
Onyx) 5.14.0-427.13.1.el9_4.x86_64			containerd://1.7.21				

troubleshooting: Pod só se comunica com pod do mesmo host!

Esse troubleshooting é referente a um problema na interface CNI do Flannel, que faz com que um Pod só consiga se conectar com outro Pod que esteja trabalhando no mesmo NODE

Criei um pod com um parametro para remover quando sair do terminal para efetuar esse teste

```
$ kubectl run --image alpine --rm -it demo sh
```

Veja que ele foi criado no worker-2

```
$ kubectl get pod -owide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE
READINESS GATES								
demo	1/1	Running	0	19m	10.244.2.18	worker-2	<none>	<none>
nginx	1/1	Running	3 (23m ago)	6d22h	10.244.1.8	worker-1	<none>	<none>
nginx01	1/1	Running	0	59s	10.244.1.9	worker-1	<none>	<none>
nginx02	1/1	Running	0	9s	10.244.2.19	worker-2	<none>	<none>

Criei outros 2 pods do nginx para montar o teste

```
$ kubectl run --image nginx nginx01 (worker-1)
```

```
$ kubectl run --image nginx nginx02 (worker-2)
```

Dentro do Pod "demo", se tentarmos pingar o Pod nginx01, que está trabalhando no NODE worker-1, não vai funcionar.

```
/ # ping 10.244.1.9
```

```
PING 10.244.1.9 (10.244.1.9): 56 data bytes
```

```
^C
```

```
--- 10.244.1.9 ping statistics ---
```

```
5 packets transmitted, 0 packets received, 100% packet loss
```

Agora se tentarmos pingar o Pod nginx02, que está trabalhando no NODE worker-2, vai funcionar

```
/ # ping 10.244.2.19
```

```
PING 10.244.2.19 (10.244.2.19): 56 data bytes
```

```
64 bytes from 10.244.2.19: seq=0 ttl=64 time=0.067 ms
```

```
64 bytes from 10.244.2.19: seq=1 ttl=64 time=0.078 ms
```

```
64 bytes from 10.244.2.19: seq=2 ttl=64 time=0.087 ms
```

```
64 bytes from 10.244.2.19: seq=3 ttl=64 time=0.084 ms
```

^C

--- 10.244.2.19 ping statistics ---

4 packets transmitted, 4 packets received, 0% packet loss

round-trip min/avg/max = 0.067/0.079/0.087 ms

E porque isso acontece?

Por que o Flannel está utilizando a interface default criada pelo Vagrant

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000

link/ether 08:00:27:fc:e9:96 brd ff:ff:ff:ff:ff:ff

altname enp0s3

inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0

Onde o correto era ele utilizar a interface de rede onde as VMs do cluster estão, que no meu caso é a 172.89.0.0/24

3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000

link/ether 08:00:27:f1:a6:1b brd ff:ff:ff:ff:ff:ff

altname enp0s8

inet 172.89.0.21/24 brd 172.89.0.255 scope global noprefixroute eth1

Para corrigir esse problema, será necessário editar o daemon-set do Flannel

Para listar os daemon-sets que estão rodando no cluster:

\$ kubectl get ds -A

NAMESPACE SELECTOR	NAME AGE	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE
kube-flannel	kube-flannel-ds	3	3	3	3	<none>	13d

```
kube-system kube-proxy 3 3 3 3 3 kubernetes.io/os=linux 13d
```

Para editar o daemon-set

```
$ kubectl edit ds -n kube-flannel kube-flannel-ds
```

E conforme o site abaixo, devemos incluir um parametro no flannel args

<https://github.com/flannel-io/flannel/issues/1098>

Que é o "--iface=INTERFACE"

- args:
- --ip-masq
- --iface=eth1
- --kube-subnet-mgr

No momento que você termina de editar, será feito o recipe, os pods antigos serão descartados para dar vida aos novos

```
$ kubectl get pod -n kube-flannel
```

NAME	READY	STATUS	RESTARTS	AGE
kube-flannel-ds-5qhk2	1/1	Running	0	34s
kube-flannel-ds-hlvwj	1/1	Running	0	37s
kube-flannel-ds-tb45w	1/1	Running	0	42s

Após isso, é possível se comunicar com um Pod em outro NODE

```
/ # ping 10.244.1.8
```

```
PING 10.244.1.8 (10.244.1.8): 56 data bytes
```

```
64 bytes from 10.244.1.8: seq=0 ttl=62 time=0.484 ms
```

```
64 bytes from 10.244.1.8: seq=1 ttl=62 time=0.393 ms
```

64 bytes from 10.244.1.8: seq=2 ttl=62 time=0.395 ms

64 bytes from 10.244.1.8: seq=3 ttl=62 time=0.346 ms

^C

--- 10.244.1.8 ping statistics ---

4 packets transmitted, 4 packets received, 0% packet loss

round-trip min/avg/max = 0.346/0.404/0.484 ms

Alterando o command do Pod

Criei um pod do ansible com alpine Linux para testar, porém, quando executei a imagem, o Pod apresentou o erro de "CrashLoopBackOff"

Isso acontece por que algumas aplicações só executam o binário e dão um exit na sequência, o Pod não fica em running disponível para uso e teste.

Mas existe uma forma de alterar o command do Pod, adicionando um "sleep" por exemplo, permitindo acessar o Pod.

Primeiro puxei a imagem com o kubectl run e coletei a output no formato yaml e fim um tratamento com o neat

```
$ kubectl run --image alpine/ansible:latest ansible-demo --dry-run -oyaml | kubectl neat
```

apiVersion: v1

kind: Pod

metadata:

labels:

run: ansible-demo

name: ansible-demo

spec:

containers:

- image: alpine/ansible:latest

name: ansible-demo

Com o yaml em mãos, editei o mesmo e ficou assim:

/tmp/ansible.yaml

apiVersion: v1

kind: Pod

metadata:

labels:

run: ansible-demo

name: ansible-demo

spec:

containers:

- image: alpine/ansible:latest

name: ansible-demo

command:

- "sleep"

- "999999999999"

Por fim, basta dar um apply

\$ kubectl apply -f /tmp/ansible.yaml

E desta vez é possível acessar o Pod!

\$ kubectl exec -it ansible-demo sh

```
/ # ansible --version
ansible [core 2.17.0]
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules',
'/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3.12/site-packages/ansible
  ansible collection location = /root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.6 (main, Sep 12 2024, 21:15:48) [GCC 13.2.1 20240309]
  (/usr/bin/python3)
  jinja version = 3.1.4
  libyaml = True
```

Init Containers

Init containers não fazem parte do processo principal do POD, é um container auxiliar que executa um job referente a inicialização da app

Por exemplo:

Antes de subir minha aplicação, vou criar um init container que ficará dando um "GET" em uma API, esperando o código de retorno 200, enquanto isso não acontecer, a aplicação não será iniciada.

Ou seja, estamos trabalhando aqui com pré-requisitos e dependências.

Vamos fazer um teste prático

Primeiro, vamos criar um pod usando a imagem do BusyBox, que é normalmente usado para um init container, pois possui os principais comandos da base Linux

```
$ kubectl run --image busybox:1.28 --rm -it busybox-demo sh
```

Com o container criado, vamos executar o comando do init container

Para exemplificar, o init container ficará executando um nslookup em um nome que não tem resolução (posteriormente iremos criar um service com o nome mymysql para ele resolver), porém nesse momento, ele ficará apresentando falha

```
$ until nslookup mymysql ; do echo "Trying to connect..." ; echo "" ; sleep 2 ; done
```

```
Server: 10.96.0.10
```

```
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local
```

```
nslookup: can't resolve 'mymysql'
```

```
Trying to connect...
```

Agora, enquanto o looping do until é executado, vamos criar um service chamado mymysql

```
$ kubectl create svc clusterip mymysql --tcp=80:80
```

```
service/mymysql created
```

No momento que o service é criado, o looping do until é finalizado, por que ele consegue resolver.

```
Server: 10.96.0.10
```

```
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local
```

```
nslookup: can't resolve 'mymysql'
```

```
Trying to connect...
```

```
Server: 10.96.0.10
```

```
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local
```

Name: mymysql

Address 1: 10.108.24.114 mymysql.default.svc.cluster.local

Essa é a ideia por trás do init container, agora vamos criar um arquivo yaml adaptado com ele

o yaml ficou dessa forma

apiVersion: v1

kind: Pod

metadata:

name: nginx-teste

namespace: default

spec:

containers:

- name: nginx

image: nginx:1.14.2

ports:

- containerPort: 80

initContainers:

- name: wait4dns

image: busybox:1.28

command:

- sh

- -c

- until nslookup mymysql; do echo "trying to resolve..." ; sleep 2 ; done

Obs: o "sh" e "-c" é necessário quando vamos passar algum comando direto no initContainer

\$ kubectl apply -f pod.yaml

```
$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
ansible-demo	1/1	Running	1 (41m ago)	4d22h
nginx	1/1	Running	5 (41m ago)	12d
nginx-teste	0/1	Init:0/1	0	9s

Quando não é atendido a condição do initContainer, o status dele fica da forma listada acima

Vamos ler os logs do initContainer

```
$ kubectl logs nginx-teste -c wait4dns
```

```
trying to resolve...
```

```
nslookup: can't resolve 'mymysql'
```

```
Server: 10.96.0.10
```

```
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local
```

```
trying to resolve...
```

```
nslookup: can't resolve 'mymysql'
```

E no momento em que criamos o serviço mymysql, o Pod é executado.

```
$ kubectl create svc clusterip mymysql --tcp=80:80
```

```
service/mymysql created
```

```
$ kubectl get pod -w
```

NAME	READY	STATUS	RESTARTS	AGE
ansible-demo	1/1	Running	1 (49m ago)	4d22h
nginx	1/1	Running	5 (49m ago)	12d
nginx-teste	0/1	Init:0/1	0	7m16s
nginx-teste	0/1	PodInitializing	0	7m58s
nginx-teste	1/1	Running	0	7m59s

Multi-container pods

É possível criar um pod com mais de um container, para isso utilizamos um arquivo yaml:

O arquivo que usei na prática:

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: multicontainer
  name: multicontainer
spec:
  containers:
    - image: httpd
      name: apache
    - image: alpine
      name: debug
      command:
        - "sleep"
        - "9999999999999999"
```

Veja que na sessão "containers", podemos colocar quantas imagens forem necessárias, nesse caso, subi um Pod com a imagem do httpd (apache) e um alpine com um comando sleep.

O importante de se notar aqui é que os containers compartilham a mesma interface/network space

Para visualizarmos isso na prática:

Acessando o pod do alpine:

```
$ kubectl exec multicontainer -it -c debug sh
```

E listando os sockets de rede

```
$ netstat -an
```

Active Internet connections (servers and established)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	:::80	:::*	LISTEN

Active UNIX domain sockets (servers and established)

Proto	RefCnt	Flags	Type	State	I-Node	Path
-------	--------	-------	------	-------	--------	------

É possível visualizar uma conexão na porta 80, que é a http e esse processo está rodando no outro container, porém, como ambos estão no mesmo pod, o container de debug também consegue ouvir essa conexão.

Lembrando que para obter logs ou executar comandos em determinado container dentro de um pod, devemos especificar com a opção "-c".

```
$ kubectl logs multicontainer
```

Defaulted container "apache" out of: apache, debug

```
$ kubectl logs multicontainer -c apache
```

```
$ kubectl logs multicontainer -c debug
```

Troubleshooting: Acessando um container de um pod sem o "kubectl exec"

Como seria possível acessar a CLI de um POD, caso não tivesse acesso ao kubectl? por meio da namespace!

Vamos lá?

Primeiro devemos saber em qual node o pod que gostaria de acessar está rodando.

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE
READINESS GATES								
ansible-demo	1/1	Running	2 (34m ago)	6d1h	10.244.2.34	worker-2	<none>	
<none>								
multicontainer	2/2	Running	2 (34m ago)	22h	10.244.2.35	worker-2	<none>	
<none>								
nginx	1/1	Running	6 (34m ago)	13d	10.244.1.21	worker-1	<none>	<none>
nginx-teste	1/1	Running	1 (34m ago)	26h	10.244.1.20	worker-1	<none>	
<none>								

No caso, vamos continuar utilizando o pod da lição anterior, que é o "multicontainer", rodando na worker-2.

Acessando o worker-2, vamos utilizar um comando para interagir com o "containerd", que é o responsável por executar os containers propriamente ditos, enquanto o kubernetes faz o papel de orquestrador.

Inclusive o crictl possui várias opções interessantes

exec	Run a command in a running container
inspect	Display the status of one or more containers
logs	Fetch the logs of a container
ps	List containers
Pods	List pods
rm	Remove one or more containers

Entre outros...

Mas o que vamos utilizar no momento, é a opção "ps", para listar os containers.

```
$ crictl ps 2> /dev/null
```

CONTAINER POD ID	IMAGE POD	CREATED	STATE	NAME	ATTEMPT
25a49da778ddc 31c0a94b52070	91ef0af61f39e multicontainer	6 minutes ago	Running	debug	0
0256f4645cfa1 31c0a94b52070	5daf6a4bfe741 multicontainer	6 minutes ago	Running	apache	0
796711739f43f 03b399de472a0	abf2c52da1b2a ansible-demo	About an hour ago	Running	ansible-demo	3
57f863d781b4c 3cf32019742ee	ead0a4a53df89 coredns-5dd5756b68-w6sp9	About an hour ago	Running	coredns	11
54c0764af1d50 f9b0be5e18d09	ead0a4a53df89 coredns-5dd5756b68-bz4qf	About an hour ago	Running	coredns	11
cc7eaba696ad8 4045c0169ee44	f7b837852a098 kube-flannel-ds-5qhk2	About an hour ago	Running	kube-flannel	4
6de930dca2146 6ebe030884c4e	31fde28e72a31 kube-proxy-pt4vn	About an hour ago	Running	kube-proxy	11

Vamos acessar o container debug, com o objetivo de rodar um "curl" no container do apache.

Para isso, precisaremos coletar o PID do container debug

```
$ crictl inspect 25a49da778ddc 2> /dev/null | grep pid
```

```
"pid": 21905,  
  "pid": 1  
  "type": "pid"
```

Ou, caso prefira uma saída mais "clean"

```
$ crictl inspect 25a49da778ddc 2> /dev/null | jq '.info.pid'
```

```
21905
```

Poderíamos utilizar o bom e velho "ps -ef | grep <command>" que também daria no mesmo, o objetivo é coletar o PID do container debud, que está rodando um sleep 9999999...

Com o PID em mãos, podemos executar comandos nesse container.

Para isso usaremos o "nsenter"

A opções usadas são:

-t, --target <pid> target process to get namespaces from (informa o PID)

-n, --net[=<file>] enter network namespace (para falar que desejo executar um comando dentro da network namespace daquele processo).

```
$ nsenter -t 21905 -n curl 10.244.2.40 -l
```

```
HTTP/1.1 200 OK
```

```
Date: Wed, 02 Oct 2024 01:50:52 GMT
```

```
Server: Apache/2.4.62 (Unix)
```

```
Last-Modified: Mon, 11 Jun 2007 18:53:14 GMT
```

```
ETag: "2d-432a5e4a73a80"
```

```
Accept-Ranges: bytes
```

```
Content-Length: 45
```

```
Content-Type: text/html
```

Sucesso!!!!

Static Pods

Antes de seguir para o assunto principal desse tópico, vamos reforçar o processo de criação normal de um Pod.

1) Quando aplicamos um pod, a requisição vai para o apiserver

2) O apiserver envia essa requisição para o kube-scheduler

3) Por fim, o kube-scheduler decide em qual node aquele pod será executado, com base nos requisitos da imagem.

Agora indo para os statics pods, que são pods que são vinculados a um node específico.

E uma das principais características desse tipo de pod é que sempre que ele for deletado, será recriado na sequência.

Isso por que, a requisição de criação do pod não passa pelo apiserver, é feito um "bypass" e quem atende a requisição é o kubectl.

E o que o kubectl faz exatamente?

Ele fica lendo o diretório /etc/kubernetes/manifests e qualquer arquivo novo (pod) criado nessa estrutura, vira um static pod.

Inclusive, o static pod é utilizado em casos muito específicos e principalmente nos pods que estão no control plane, todos os pods são estáticos!

Lifecycle: Como um pod [e terminado?

Quando deletamos um pod, um sinal é enviado ao processo desse pod, e o sinal enviado é um SIGTERM (15).

A aplicação que está rodando dentro do pod, deve ter um mecanismo que receba e entenda esse sinal, para começar a desligar os componentes da app da melhor forma possível.

Caso a aplicação não entenda esse sinal, será iniciará um timeout de 30 segundos (padrão) e após esse tempo, será enviado um SIGKILL(9) para o processo, terminando ele abruptamente.

É possível personalizar esse processo, adicionando mais tempo no timeout, configurando uma ação "pré-stop", onde a app irá executar esse comando quando receber o SIGTERM.

Arquivo YAML com essas configurações:

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: worker-pod
  name: worker-pod
spec:
  terminationGracePeriodSeconds: 60
  containers:
    - image: alpine
      name: alpine
      command:
        - "sleep"
        - "99999999"
      lifecycle:
        preStop:
          exec:
            command:
              - sh
              - -c
              - curl 10.244.2.32
```

terminationGracePeriodSeconds

Opção usada para adicionar mais tempo para a aplicação executar os comandos para parar os componentes.

lifecycle e preStop

Opções para configurar uma ação a ser realizada pela app quando receber um SIGTERM

Introdução aos deployments

Normalmente a utilização de pods é para realizar testes e troubleshootings.

Quando vamos expor a aplicação em um ambiente mais crítico, é utilizado uma das 3 formas abaixo:

- Deployments
- Statefull set
- Daemon set

Onde o deployment é utilizado para aplicações stateless (basicamente são aplicações que conseguimos escalar horizontalmente)

Statefull set, o próprio nome diz (Uma aplicação statefull, devemos ter mais cuidado ao escalar, geralmente essas aplicações possuem parametros já estabelecidos)

E o Daemon set, vai colocar 1 pod em cada node do cluster (normalmente aplicações coletoras de logs, por exemplo)

Agora explorando um pouco mais sobre o deployment:

Quando criamos um Deployment, embaixo dos panos ele criará um "replica set", que por sua vez irá definir o número de replicas que serão utilizados.

Deployment -> Replica set -> Pods

Para criar um deployment, utilizamos o `kubectl create`

A partir daqui podemos aplicar aquelas técnicas aprendidas anteriormente... usando o `-oyaml`, `--dry-run`, `kubectl neat...`

```
$ kubectl create deployment --image nginx web-server --dry-run -oyaml | kubectl neat > nginx.yaml
```

```
$ kubectl apply -f nginx.yaml
```

Para listar os deployments

```
$ kubectl get deploy
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
web-server	1/1	1	1	9s

Deployment -> Replica set -> Pods

Explorando um pouquinho mais esse processo de criar um deployment, temos a criação de um replica set, e para listar no cluster, usamos:

```
$ kubectl get replicaset
```

Uma coisa interessante de se notar é a estrutura do nome dos pods quando são criados através de um deployment

NAME	READY	STATUS	RESTARTS	AGE
web-server-56fdb95f7f-pnxh9	1/1	Running	0	5s
web-server-56fdb95f7f-tl87x	1/1	Running	0	7s

Temos:

web-server (deployment)

56fdb95f7f (ID do replica set)

pnxh9 (ID do pod)

Os ID's do replica set e pod são criados de forma aleatória!

Agora entendendo um pouco o processo de deploy, onde é usado o "rolling update", que é basicamente definido uma porcentagem de Pods que vão ser destruídos no antigo deployment e criados no novo.

Ao editar o deployment e mudando a imagem de nginx para httpd, por exemplo, será criado um novo replica set, onde ele vai deletar um pod do antigo replica set e criar no novo.

```
$ kubectl edit deploy web-server  
deployment.apps/web-server edited
```

```
$ kubectl get po
```

NAME	READY	STATUS	RESTARTS	AGE
web-server-56fdb95f7f-tl87x	0/1	ContainerCreating	0	2s
web-server-6484c4f547-95snj	1/1	Running	0	87s
web-server-6484c4f547-z6xcx	1/1	Running	0	24m

```
$ kubectl get replicaset
```

NAME	DESIRED	CURRENT	READY	AGE
web-server-56fdb95f7f	2	2	2	2m33s
web-server-6484c4f547	0	0	0	26m

```
## kubectl explain: Obtendo documentação
```

Um comando muito interessante que pode ser utilizado para entender melhor as apis do kubernetes é o "kubectl explain"

```
$ kubectl explain deployment
```

```
$ kubectl explain configmap
```

```
$ kubectl explain statefulsets
```

```
$ kubectl explain daemonsets
```

Entre outros...

É comum usar o explain quando queremos saber o que podemos colocar em determinados campos da estrutura do YAML

Exemplo:

O que posso utilizar dentro da chave "metadata"?

```
$ kubectl explain deployment.metadata
```

Posso mudar a estratégia de deploy no meu deployment?

```
$ kubectl explain deployment.spec.strategy
```

```
$ kubectl explain deployment.spec.template
```

Template describes the pods that will be created.

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  labels:
```

```
    app: web-server
```

```
    environment: hint
```

```
  name: web-server
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: web-server
```

```
  template:
```

metadata:

creationTimestamp: null

labels:

app: web-server

environment: hint

spec:

containers:

- image: nginx

name: nginx

Gerenciando rollouts

Explorando um pouco o kubectl rollout, comando utilizando para gerenciar o rollout de varios recursos

Os recursos válidos são:

- deployments
- daemonsets
- statefulsets

Opções:

- | | |
|---------|--------------------------------------|
| history | View rollout history |
| pause | Mark the provided resource as paused |
| restart | Restart a resource |
| resume | Resume a paused resource |
| status | Show the status of the rollout |
| undo | Undo a previous rollout |

Exemplos:

Lista o histórico de rollout de um deployment

```
$ kubectl rollout history deployment/web-server
```

```
deployment.apps/web-server
```

```
REVISION  CHANGE-CAUSE
```

```
1      <none>
```

```
2      <none>
```

Acompanha o status de um rollout

```
$ kubectl rollout status deployment/web-server
```

```
Waiting for deployment "web-server" rollout to finish: 3 out of 5 new replicas have been updated...
```

```
Waiting for deployment "web-server" rollout to finish: 3 out of 5 new replicas have been updated...
```

```
Waiting for deployment "web-server" rollout to finish: 3 out of 5 new replicas have been updated...
```

```
Waiting for deployment "web-server" rollout to finish: 3 out of 5 new replicas have been updated...
```

```
Waiting for deployment "web-server" rollout to finish: 3 out of 5 new replicas have been updated...
```

```
Waiting for deployment "web-server" rollout to finish: 4 out of 5 new replicas have been updated...
```

```
Waiting for deployment "web-server" rollout to finish: 4 out of 5 new replicas have been updated...
```

```
Waiting for deployment "web-server" rollout to finish: 4 out of 5 new replicas have been updated...
```

```
Waiting for deployment "web-server" rollout to finish: 2 old replicas are pending termination...
```

```
Waiting for deployment "web-server" rollout to finish: 2 old replicas are pending termination...
```

```
Waiting for deployment "web-server" rollout to finish: 2 old replicas are pending termination...
```

```
Waiting for deployment "web-server" rollout to finish: 1 old replicas are pending termination...
```

```
Waiting for deployment "web-server" rollout to finish: 1 old replicas are pending termination...
```

```
Waiting for deployment "web-server" rollout to finish: 1 old replicas are pending termination...
```

Waiting for deployment "web-server" rollout to finish: 4 of 5 updated replicas are available...
deployment "web-server" successfully rolled out

Para voltar para a revisão anterior

\$ kubectl rollout undo deployment/web-server

Ele vai voltar para o replicaset "web-server-79796ffdff"

[root@master-1 tmp]# kubectl get replicaset

NAME	DESIRED	CURRENT	READY	AGE
web-server-55b7f6dd8c	5	5	5	6m33s
web-server-79796ffdff	0	0	0	4d23h

[root@master-1 tmp]# kubectl get pod

NAME	READY	STATUS	RESTARTS	AGE
web-server-55b7f6dd8c-22rzz	1/1	Running	0	9m31s
web-server-55b7f6dd8c-7xvz	1/1	Running	0	9m25s
web-server-55b7f6dd8c-ljsjp	1/1	Running	0	9m31s
web-server-55b7f6dd8c-xw82j	1/1	Running	0	9m24s
web-server-55b7f6dd8c-z6lq5	1/1	Running	0	9m31s

Depois de executar o undo...

[root@master-1 tmp]# kubectl get replicaset

NAME	DESIRED	CURRENT	READY	AGE
web-server-55b7f6dd8c	0	0	0	10m
web-server-79796ffdff	5	5	5	4d23h

[root@master-1 tmp]# kubectl get pod

NAME	READY	STATUS	RESTARTS	AGE
web-server-79796ffdff-4hbcx	1/1	Running	0	21s

web-server-79796ffdff-4tnpf	1/1	Running	0	19s
web-server-79796ffdff-d57gq	1/1	Running	0	21s
web-server-79796ffdff-m9bm7	1/1	Running	0	20s
web-server-79796ffdff-pkz2z	1/1	Running	0	21s

É possível retornar para uma revisão específica:

```
[root@master-1 tmp]# kubectl rollout history deployment/web-server
deployment.apps/web-server
```

REVISION CHANGE-CAUSE

```
2    <none>
4    <none>
5    <none>
6    <none>
```

```
$ kubectl rollout undo deployment/web-server --to-revision=2
deployment.apps/web-server
```

```
[root@master-1 tmp]# kubectl get rs
```

NAME	DESIRED	CURRENT	READY	AGE
web-server-548fccb654	0	0	0	14m
web-server-55b7f6dd8c	4	4	4	31m
web-server-776db4f4f4	0	0	0	7m4s
web-server-79796ffdff	0	0	0	5d

```
[root@master-1 tmp]#
```

```
[root@master-1 tmp]#
```

```
[root@master-1 tmp]# kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
web-server-55b7f6dd8c-fzskk	1/1	Running	0	117s
web-server-55b7f6dd8c-qplqn	1/1	Running	0	117s
web-server-55b7f6dd8c-rkwrs	1/1	Running	0	115s

web-server-55b7f6dd8c-rsplp 1/1 Running 0 115s

Como saber o revision atual:

Pegue o replicaset atual e execute um describes

```
$ kubectl describe rs web-server-55b7f6dd8c | grep revision
```

```
deployment.kubernetes.io/revision: 7
```

```
deployment.kubernetes.io/revision-history: 2
```

maxSurge e maxUnavailable: Otimizando deployment

Os parametros maxSurge e maxUnavailable são utilizados para otimizar ou deixar o deploy mais estável (com menos downtime e menos velocidade, consequentemente).

O maxSurge é o parametro que indica quantos pods serão criados acima do número desejado durante o processo de update.

Por exemplo, se você configurou 100 réplicas, durante o update, será criado +25 novos pods, indo para 125 e no mesmo momento, 25 pods do antigo replicaset serão destruídos e assim até finalizar o deploy.

Já o maxUnavailable, indica a quantidade de pods que podem ficar indisponíveis "Terminating" durante o processo de update.

Por exemplo, se você configurar o valor "0", os pods do antigo replicaset serão destruídos apenas quando todos os novos pods forem criados.

Essa opção é muito interessante, por que ela diminui o downtime da aplicação, em custo da velocidade do deploy.

Caso algum pod do antigo replicaset, durante um rollingUpdate esteja recebendo uma requisição, não corre o risco dele ser destruído.

Os valor padrão para ambos os parâmetros é de 25%.

Segue o exemplo de como fica o bloco no yaml referente a essas configurações

spec:

strategy:

rollingUpdate:

maxSurge: 10%

maxUnavailable: 0

Escalando um deployment

Até o momento, aprendemos apenas uma forma de definir a quantidade de pods em um deployment, que é alterando o yaml ou editando o deploy

```
$ kubectl edit deploy
```

Porém, existe uma forma mais simples de efetuar esse processo.

Utilizando o comando "kubectl scale"

Vamos aos exemplos:

Diminuindo a quantidade de replicas de um deployment pela metade (10 -> 5)

```
$ kubectl get deploy
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
web-server	10/10	10	10	5d23h

```
$ kubectl scale deployment web-server --replicas=5
```

```
deployment.apps/web-server scaled
```

```
$ kubectl get deploy
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
web-server	5/5	5	5	5d23h

!!! OBS !!!

Embora o método seja extremamente prático, ainda sim não é o mais recomendado, a melhor forma de realizar o scale up/down de recursos no kubernetes, ainda será abordado mais a frente, que é o "Horizontal Pod Autoscale".

Resources: requests e limits

Nessa seção vamos abordar as formas de delimitar a quantidade de recursos computacionais que um container poderá utilizar.

Existem dois parametros que podem ser usados para esse fim, que é o "requests" e o "limits", vamos entender a diferença entre eles:

requests:

O requests é onde definimos os valores computacionais básicos de cpu e memória que o pod em teoria precisará para executar a aplicação da melhor forma, como se fosse um pré-requisito.

Se configurarmos o requests da seguinte maneira:

spec:

containers:

- image: httpd

name: httpd

resources:

requests:

cpu: 1

memory: 4G

Quando aplicarmos a configuração, será enviado para o Schedule, que irá verificar qual node do cluster pode atender esses requisitos, caso nenhum node consiga oferecer esses recursos computacionais

O container ficará em "Pending" por insufficient memory ou cpu.

Mas caso exista um node que consiga comportar esses requisitos, o schedule irá colocar o pod para executar nesse node.

limits:

O limits diz respeito ao valor computacional máximo que um pod irá utilizar, não podendo ultrapassar esse limite.

spec:

containers:

- image: httpd

name: httpd

resources:

requests:

cpu: 1

memory: 4G

limits:

cpu: 10

memory: 16G

Agora em um cenário onde o pod é criado em um node que possui "3 vcpus" e 6GB de memória, pelos valores do requests, ele é aceito.

Porém, observando os valores do limits, colocamos números elevados demais para o node em questão, dessa forma, o limits do pod será o limite do node (3cpu e 6Gb mem).

Como analisar um OOM (Out Of Memory) no cluster Kubernetes

O Out Of Memory ocorre quando o Kernel Linux identifica que o container (processo) está consumindo muita memória.

Quando ocorre o OOM, devemos analisar no NODE onde o pod está executando.

Criei um deployment que utiliza uma imagem docker com java, que ficará alocando memória de forma repetitiva até apresentar o OOM

Segue o yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: oom-deployment
  name: oom-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: oom-deployment
  template:
    metadata:
      labels:
        app: oom-deployment
    spec:
      containers:
        - image: valentinomiazzo/jvm-memory-test
          name: memorytest
          env:
```

```
- name: "ALLOC_HEAP_MB"
  value: "100"
- name: "MAX_HEAP_SIZE_MB"
  value: "2000"
resources:
  requests:
    cpu: 100m
    memory: 512M
  limits:
    cpu: 1
    memory: 1G
```

Logo após aplicar o deployment, apresentou o status OOMKilled

```
$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
oom-deployment-6876764645-cprdv	0/1	OOMKilled	0	12s
web-server-6b5654cfd9-89l8w	1/1	Running	2 (169m ago)	4d14h

Após identificar esse cenário, o próximo passo é identificar o node ao qual esse pod está executando

```
$ kubectl describe pod oom-deployment-84f86c696f-nz5sf | grep Node
```

```
Node:          worker-2/172.89.0.22
```

No arquivo /var/log/messages é possível identificar mensagens de erro OOM

```
Oct 28 15:15:59 worker-2 kernel: Out of memory: Killed process 55164 (java) total-
vm:3394048kB, anon-rss:1656420kB, file-rss:128kB, shmem-rss:0kB, UID:0 pgtables:3376kB
oom_score_adj:876
```

A partir desse momento, o ideal é parar a aplicação e rever a quantidade de memória necessária para comportar a aplicação.

Readiness vs Liveness Probes

readiness: kubelet usa o readiness probes para saber quando um container (pod) está pronto (read -> "ready") para iniciar e receber o tráfego.

liveness: kubelet usa o liveness probes para saber quando reiniciar um container. Por exemplo, o liveness probe pode pegar um deadlock no pod, por travament, OOM e etc.

O restart pode ajudar a aplicação voltar a ficar disponível "live".

E as verificações são realizadas geralmente executando o conceito de "Healthchecks", esperando um código de retorno 200/300 para sucesso e diferente disso para erro.

A configuração do readiness e liveness probe fica na sessão containers na estrutura de um arquivo yaml.

Exemplo utilizando um http get

```
spec:
  containers:
  - image: nginx
    name: nginx
    readinessProbe:
      httpGet:
        path: "/"
        port: 80
    livenessProbe:
      httpGet:
        path: "/"
        port: 80
```

Após realizar um kubectl apply no yaml com essa configuração, o pod irá subir, e será possível observar a consulta http get sendo realizada a cada 10s (padrão)

```
$ kubectl logs nginx-web-b9dbf666-c44c4 -f
```

```
10.244.2.1 - - [29/Oct/2024:15:53:51 +0000] "GET / HTTP/1.1" 200 615 "-" "kube-probe/1.28"
"_"
```

```
10.244.2.1 - - [29/Oct/2024:15:54:01 +0000] "GET / HTTP/1.1" 200 615 "-" "kube-probe/1.28"
"_"
```

Agora se acessarmos o pod e deletarmos o arquivo padrão que é utilizado pelo nginx para requisição get "/usr/share/nginx/html/index.html"

```
$ kubectl exec -it nginx-web-b9dbf666-c44c4 -f sh
```

```
$ rm -f /usr/share/nginx/html/index.html
```

Começamos a receber um código de erro 403

```
$ kubectl logs nginx-web-b9dbf666-c44c4 -f
```

```
2024/10/29 15:57:31 [error] 29#29: *47 directory index of "/usr/share/nginx/html/" is
forbidden, client: 10.244.2.1, server: localhost, request: "GET / HTTP/1.1", host:
"10.244.2.101:80"
```

```
10.244.2.1 - - [29/Oct/2024:15:57:31 +0000] "GET / HTTP/1.1" 403 153 "-" "kube-probe/1.28"
"_"
```

```
2024/10/29 15:57:31 [error] 29#29: *46 directory index of "/usr/share/nginx/html/" is
forbidden, client: 10.244.2.1, server: localhost, request: "GET / HTTP/1.1", host:
"10.244.2.101:80"
```

```
10.244.2.1 - - [29/Oct/2024:15:57:31 +0000] "GET / HTTP/1.1" 403 153 "-" "kube-probe/1.28"
"_"
```

```
2024/10/29 15:57:41 [error] 29#29: *49 directory index of "/usr/share/nginx/html/" is
forbidden, client: 10.244.2.1, server: localhost, request: "GET / HTTP/1.1", host:
"10.244.2.101:80"
```

```
10.244.2.1 - - [29/Oct/2024:15:57:41 +0000] "GET / HTTP/1.1" 403 153 "-" "kube-probe/1.28"
"_"
```

```
2024/10/29 15:57:41 [error] 29#29: *48 directory index of "/usr/share/nginx/html/" is
forbidden, client: 10.244.2.1, server: localhost, request: "GET / HTTP/1.1", host:
"10.244.2.101:80"
```

```
10.244.2.1 - - [29/Oct/2024:15:57:41 +0000] "GET / HTTP/1.1" 403 153 "-" "kube-probe/1.28"
"_"
```

```
2024/10/29 15:57:51 [error] 29#29: *50 directory index of "/usr/share/nginx/html/" is
forbidden, client: 10.244.2.1, server: localhost, request: "GET / HTTP/1.1", host:
"10.244.2.101:80"
```

```
2024/10/29 15:57:51 [error] 29#29: *51 directory index of "/usr/share/nginx/html/" is
forbidden, client: 10.244.2.1, server: localhost, request: "GET / HTTP/1.1", host:
"10.244.2.101:80"
```

```
10.244.2.1 - - [29/Oct/2024:15:57:51 +0000] "GET / HTTP/1.1" 403 153 "-" "kube-probe/1.28"
"_"
```

```
10.244.2.1 - - [29/Oct/2024:15:57:51 +0000] "GET / HTTP/1.1" 403 153 "-" "kube-probe/1.28"
"_"
```

```
2024/10/29 15:57:51 [error] 29#29: *52 directory index of "/usr/share/nginx/html/" is
forbidden, client: 10.244.2.1, server: localhost, request: "GET / HTTP/1.1", host:
"10.244.2.101:80"
```

```
10.244.2.1 - - [29/Oct/2024:15:57:51 +0000] "GET / HTTP/1.1" 403 153 "-" "kube-probe/1.28"
"_"
```

```
2024/10/29 15:57:51 [error] 29#29: *53 directory index of "/usr/share/nginx/html/" is
forbidden, client: 10.244.2.1, server: localhost, request: "GET / HTTP/1.1", host:
"10.244.2.101:80"
```

```
10.244.2.1 - - [29/Oct/2024:15:57:51 +0000] "GET / HTTP/1.1" 403 153 "-" "kube-probe/1.28"
"_"
```

```
2024/10/29 15:57:51 [notice] 1#1: signal 3 (SIGQUIT) received, shutting down
```

```
2024/10/29 15:57:51 [notice] 29#29: gracefully shutting down
```

```
2024/10/29 15:57:51 [notice] 29#29: exiting
```

```
2024/10/29 15:57:51 [notice] 29#29: exit
```

```
2024/10/29 15:57:51 [notice] 1#1: signal 17 (SIGCHLD) received from 29
```

```
2024/10/29 15:57:51 [notice] 1#1: worker process 29 exited with code 0
```

```
2024/10/29 15:57:51 [notice] 1#1: exit
```

E após 3 health checks de 10s (padrão, é possível alterar isso), o pod é restartado pelo liveness probe!

```
$ kubectl get pod -w
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-web-b9dbf666-c44c4	1/1	Running	0	74s

web-server-6b5654cfd9-89l8w	1/1	Running	3 (149m ago)	5d16h
nginx-web-b9dbf666-c44c4	0/1	Running	0	4m10s
nginx-web-b9dbf666-c44c4	0/1	Running	1 (2s ago)	4m12s
nginx-web-b9dbf666-c44c4	1/1	Running	1 (2s ago)	4m12s

Introdução aos DaemonSets

DaemonSets são recursos do Kubernetes que garante que um determinado Pod seja executado em todos os nodes do cluster.

Não é possível definir a quantidade de replicas, por que ele vai garantir a execução de um (1) único pod em cada node.

Isso é útil para serviços como:

- Monitoramento de desempenho
- Coletor de logs
- CNI (Flannel, calico)

Basicamente, o que for necessário executar a nível do host, utiliza-se os DaemonSets

Assim como no deployment ou StatefulSets, podemos utilizar de tags para definir em que node a aplicação irá executar.

Por exemplo, no ambiente, podemos ter nodes com mais GPU para aplicações de Machine Learning e etc

Então ao invés de executar o DaemonSet em todos os nodes do cluster, é possível colocar em apenas um grupo do cluster.

Vamos praticar:

```
$ kubectl get node --show-labels
```

NAME	STATUS	ROLES	AGE	VERSION	LABELS
master-1	Ready	control-plane	38h	v1.28.15	beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=master-1,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=,node.kubernetes.io/exclude-from-external-load-balancers=
worker-1	Ready	<none>	38h	v1.28.15	beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=worker-1,kubernetes.io/os=linux
worker-2	Ready	<none>	38h	v1.28.15	beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=worker-2,kubernetes.io/os=linux

Cada node possui uma label que especifica o "hostname", utilizando ela junto a opção "nodeSelector", podemos lançar o pod somente em um node específico.

```
kubernetes.io/hostname=<NODE>
```

Assim ficou o YAML:

```
apiVersion: apps/v1
```

```
kind: DaemonSet
```

```
metadata:
```

```
  labels:
```

```
    app: nginxds
```

```
  name: nginxds
```

```
spec:
```

```
  selector:
```

```
    matchLabels:
```

```
      app: nginxds
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: nginxds
```

```
spec:
  containers:
  - image: nginx
    name: nginx
  nodeSelector:
    kubernetes.io/hostname: worker-1
```

Temos um único pod rodando no worker-1, conforme especificado.

```
$ kubectl get pod -owide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE
READINESS GATES							
nginxds-jw2z9	1/1	Running	1 (13m ago)	24h	10.244.1.5	worker-1	<none>
<none>							

StatefulSets

Qual a diferença entre Stateful vs Stateless?

Stateful: Armazena informações de interações anteriores para criar uma continuidade na experiência do usuário.

Por exemplo, um usuário acessa uma página Web, a sua sessão será gravada e na próxima vez que esse mesmo usuário acessar ou atualizar a página,

ele será direcionado para o mesmo node em que sua sessão foi criada.

Stateless: Cada solicitação é tratada de forma independente e isolada, não carrega os dados de interações anteriores;

Por exemplo: O usuário acessa uma aplicação que informa sobre o clima/tempo, cada requisição retornará apenas os dados atuais, sem a necessidade de lembrar a sessão anterior do usuário.

Para se trabalhar com cluster kubernetes, o mais indicado é utilizar aplicações "Stateless", pois como cada componente é tratado de forma isolada, isso permite

que ocorra a escalabilidade horizontal, aumentando o número de réplicas para atender a demanda.

Em contra partida, as aplicações Stateful possuem uma maior dificuldade de escalarem horizontalmente, por que possuem muitas dependências em suas aplicações, como replicação de dados e etc.

Exemplos de aplicações Stateless e Stateful:

Stateful:

Servidores de banco de dados, como MySQL e PostgreSQL, mantêm estado durante uma transação.

Por exemplo, uma transação de pagamento requer que o banco de dados "lembre" das operações anteriores na mesma sessão para garantir consistência nos dados.

Stateless:

O DNS (Domain Name System) é um exemplo de serviço stateless. Cada consulta a um servidor DNS é independente, e ele não precisa "lembrar" de consultas anteriores.

Introdução aos statefulsets

No StatefulSet, temos 3 features importantes que são interessantes de avaliar se uma aplicação precisa, o que vai indicar que a app de fato é um StatefulSet.

1) Deployment controlado:

Assim como vimos anteriormente, aplicações Stateful possuem uma maior dificuldade de escalar horizontalmente, mas isso não significa que seja impossível

No StatefulSet, o deployment é realizado de forma extremamente controlada, respeitando as dependências necessárias de cada aplicação e arquitetura

O deploy é feito de forma controlada, um por vez, diferentemente do deployment que podem lançar dezenas de pods.

2) Nomes previsíveis

Na criação de um recurso StatefulSet, os nomes serão previsíveis, seguindo uma ordem numérica, diferentemente de um deployment, em que os pods ganham um nome aleatório.

Exemplo:

nginx-0

nginx-1

nginx-2

...

3) Volume por pod

Quando usamos um deployment, os pods compartilham um único volume (PVC - PersistentVolumeClaim).

No StatefulSet, existe a possibilidade de cada pod possuir o seu próprio PVC, dessa forma, se um pod for destruído, os dados ainda serão persistidos.

```
$ kubectl drain <node>
```

Comando para drenar um node e impedir que novos pods sejam criados nele.

```
$ kubectl uncordon <node>
```

Comando para reverter o "drain", tornando o node elegível para receber novos pods

```
$ kubectl expose deployment nginx --port=80 --target-port=80
```

```
$ kubectl expose deployment httpd --port=80 --target-port 80 --type ClusterIP
```

Comandos para expor um service

Service discovery DNS - <https://kubernetes.io/docs/concepts/services-networking/service/#dns>

Quando tentamos resolver o nome de um service que está na mesma namespace, geralmente somente com o nome do serviço é o suficiente.

Mas caso esteja em outra namespace, é necessário resolver com a seguinte estrutura de nome: <service>.<namespace>.svc.cluster.local (ex: nginx.default.svc.cluster.local)

Como uma requisição chega nos pods?

Normalmente, uma requisição chega em um pod através dos Selectors -> LABELS

É por essas labels que o Kubernetes cria os endpoints dinamicamente e encaminha as requisições

```
$ kubectl label pod demo app=nginx
```

Adiciona uma label de um pod

```
$ kubectl get endpoints
```

Comando que mostra os endpoints do services

É muito importante prestar atenção nas labels, por que se configurar com o nome errado ou adicionar uma mesma label para mais de uma aplicação, pode resultar em problemas.

IPVS x IPTABLES

No Kubernetes, o iptables e o IPVS (IP Virtual Server) são duas maneiras de implementar o balanceamento de carga de serviços.

Ambos podem ser utilizados para rotear tráfego entre os pods e disponibilizar os serviços

IPTABLES

No Kubernetes, ele é usado para rotear o tráfego entre pods por meio de regras NAT (Network Address Translation) quando o kube-proxy está configurado para o modo iptables.

O kube-proxy configura regras no iptables para cada serviço. Quando uma solicitação é feita para um ClusterIP, NodePort ou LoadBalancer,

as regras iptables redirecionam o tráfego para um dos pods que compõem o serviço, usando uma regra de round-robin para balanceamento.

Como o número de serviços aumenta, o número de regras iptables também cresce, o que pode reduzir o desempenho em clusters de grande escala.

IPVS

O kube-proxy em modo IPVS usa tabelas específicas do kernel para rotear tráfego de rede, oferecendo uma solução de roteamento em nível de kernel mais eficiente e performática que o iptables.

IPVS é mais rápido e escalável, sendo ideal para clusters de grande escala.

```
$ k get cm -n kube-system kube-proxy -o yaml | grep mode
```

Comando para verificar se o modo de operação está configurado como "ipvs" ou "iptables"

```
k get csr estagiario-csr -ojson | jq -r '.status.certificate' | base64 -d | openssl x509 -text
```

Configuração de VirtualHost do NGINX via ConfigMap

Vamos fazer um exemplo de utilização do ConfigMap para injetar a configuração de Virtual Host dentro do pod do NGINX.

Aqui está a configuração para você utilizar também:

```
server {  
    listen 80;  
    listen [::]:80;  
  
    root /usr/share/nginx/app1;  
    index index.html index.htm index.nginx-debian.html;  
  
    server_name app.localhost.com www.app.localhost.com;  
  
    location / {  
        try_files $uri $uri/ =404;  
    }  
}
```

```
}
```

Secret Types

O Secret nada mais é do que um ConfigMap encodado em base64. Não há nada de encriptação nos secrets.

Existem vários tipos de secrets que fazem o enforcing de campos informados conforme você pode conferir <https://kubernetes.io/docs/concepts/configuration/secret/#secret-types>

Introdução a Storage no Kubernetes

Os containers são voláteis e foram feitos para "morrer" e "renascer" em um ambiente distribuído. Ainda assim, temos recursos para persistir informações e compartilhar entre os pods da aplicação, se assim desejar.

Há basicamente duas formas: Estático ou Dinâmico. Vamos entender as diferenças e começar a praticar.

Static Volumes

Vamos começar com o mais simples: Static Provisioning. Isso envolve a criação de um Persistent Volume, Persistent Volume Claim e posteriormente a montagem dentro da aplicação.

Neste exemplo usando o local para criar um PV e PVC.

Dynamic Volume

A melhor forma de ter Storage dentro do cluster na minha visão: Dynamic Volumes.

Neste caso você vai sempre precisar de uma StorageClass e um PVC para fazer a requisição.

Na aula utilizei o local-path-provisioner.

Access Modes

O Access Mode é um parâmetro obrigatório na criação do PV e PVC, divididos em:

ReadWriteOnce

ReadOnlyMany

ReadWriteMany

Lembrando que essa configuração é praticamente um best-effort. Por mais que você informe um RWO, ele pode fazer o "upgrade" para RWM

Reclaim Policy

A Reclaim Policy dita o que acontece depois de deletar um PVC. Hoje temos duas suportadas:

Delete

Retain

O Delete é o default para a grande maioria dos StorageClasses, que vai deletar o underlying storage assim que o PVC for deletado. Vai tudo pro espaço mesmo.

É possível, se quiser, mudar para Retain para reter os dados quando o PVC for deletado. O PV continua lá.

Conforme a aula, nós deletamos e depois recuperamos os dados criando um PVC apontando para o PV, sem uso de StorageClass, similar ao provisionamento estático.

Projected Volumes

Os Projected Volumes foram criados para facilitar a montagem de Secrets e ConfigMaps basicamente. É possível agrupar objetos em um único Volume, e então ter uma única diretiva de VolumeMounts no manifesto.

Teoria: HPA, kube-controller e metrics APIs

Os comportamentos default do HPA são controlados pelo kube-controller. Você pode filtrar por `--horizontal-pod-autoscaler`.

kube-controller-manager

per-pod resource metrics → A forma default calculando a média do uso dos recursos de compute do pod como um todo (mesmo tendo mais de um container).

per-pod custom metrics → Pode ser integrado com Prometheus para uso de outras métricas.

Essas são as APIs utilizadas pelo HPA:

metrics.k8s.io → A principal, implementada pelo metrics-server.

custom.metrics.k8s.io

external.metrics.k8s.io

Ou seja, é necessário fazer o deploy do metrics-server:

<https://github.com/kubernetes-sigs/metrics-server>

HPA na prática

apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: nginx-hpa

spec:

metrics:

- resource:

name: cpu

target:

averageUtilization: 60

type: Utilization

type: Resource

```
minReplicas: 1
  maxReplicas: 100
scaleTargetRef:
  apiVersion: apps/v1
  kind: Deployment
  name: nginx
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: nginx
  name: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx
        name: nginx
        resources:
          requests:
            cpu: 10m
          limits:
            cpu: 100m
```

apiVersion: v1

kind: Service

metadata:

labels:

app: nginx

name: nginx

spec:

ports:

- port: 80

selector:

app: nginx

type: ClusterIP

Simulando scaling e entendendo o algoritmo

Existe um algoritmo que é usado para definir se será escalado ou não. Por exemplo:

$$\text{desiredReplicas} = \text{ceil}[\text{currentReplicas} * (\text{currentMetricValue} / \text{desiredMetricValue})]$$

Primeiro vamos entender a função de ceil.

💡 The ceil() function computes the smallest integer that is greater than or equal to x.

Partindo do princípio que X seja 10.2, o valor será 11.

currentReplicas é, obviamente, o número de replicas atual do deployment.

currentMetricValue é a relação do uso atual em comparação aos limites definidos no requests dentro do resources.

desiredMetricValue é o ponto de escala que foi definido. No nosso exemplo foi 60%.

Vamos simular um cenário...

Nosso requests é de 10m (10 millicpus).

Rodando requisições em loop, o consumo do pod vai para 63m.

Com uma regra de 3... $(100 * 63) / 10 = 630\%$.

Aí é só aplicar a formula... $\text{ceil}[1 * (630 / 60)] = 11$

O deployment vai ser escalado para ~11 replicas.

Para simular...

```
$ kubectl run -it --image alpine demo sh
```

```
# apk add curl
```

```
# while true; do curl <svc>; done
```

Stabilization Window

Usado para evitar que os pods flutuem demais escalando e “deescalando”. Por default, depois que o pod escalar, vai demorar mais ou menos 5 minutos para estabilizar a janela, até que então parte deles possa ser deescalado.

É possível alterar a janela default no kube-controller ou sobrescrever o valor no deployment.

Horizontal Pod Autoscaling

Default values

Esses são os valores para o HPA caso não sejam modificados:

behavior:

scaleDown:

stabilizationWindowSeconds: 300

policies:

- type: Percent

value: 100

periodSeconds: 15

scaleUp:

stabilizationWindowSeconds: 0

policies:

- type: Percent

value: 100

periodSeconds: 15

- type: Pods

value: 4

periodSeconds: 15

selectPolicy: Max

Isso significa que...

Após a stabilization window, o deployment pode ser deescalado em 100% após o periodo de 15s.

Não há stabilization window para escalar.

Deve escalar de 4 em 4 pods a cada 15s. Então se no exemplo o número de replicas era 11, ele faria 1 → 5 → 15s → 9 → 15s → 11.

Ou, vai escalar 100% dos pods atuais em 15s (se tiver 1, escala somente 1).

Alterar comportamento do scale down

behavior:

scaleDown:

policies:

- type: Percent

value: 10

periodSeconds: 60

Introdução e Deploy do VPA

autoscaler/vertical-pod-autoscaler at master · kubernetes/autoscaler

O deploy do VPA é bem simples:

```
$ git clone https://github.com/kubernetes/autoscaler
```

```
$ git checkout <tag>
```

```
$ cd autoscaler/vertical-pod-autoscaler
```

```
$ ./hack/vpa-up.sh
```

Você verá 3 componentes: admission, updater e recommender (mais sobre a arquitetura aqui).

admission → Recebe os hooks quando um pod é criado e faz o patch nos request/limits.

updater → Atualiza os pods em tempo real fazendo o evicting.

recommender → Verifica a utilização de recursos e atualiza os VPAs.

Simulando scaling do VPA

Para entender bem, vamos aplicar o exemplo.

```
$ kubectl apply -f autoscaler/vertical-pod-autoscaler/examples/hamster.yaml
```

```
$ kubectl get po -w
```

```
$ kubectl get vpa -w
```

```
$ kubectl describe vpa <vpa>
```

O segredo por trás do VPA é: tanto o deployment quanto replicaset não serão editados.

No momento que um pod é criado, há um mutatingwebhook que envia para o admission-controller, e então ocorre a mudança dos requests.

```
$ kubectl get mutatingwebhookconfigurations -A
```

```
$ kubectl get endpoints -n kube-system vpa-webhook
```

```
$ kubectl logs -n kube-system <admission pod> -f
```

E mesmo após a criação do pod, ele será atualizado por meio do updater.

Introdução a CNI (Container Networking Interface)

CNI é um acrônimo para Container Network Interface, que por sua vez, é justamente a implementação da parte de rede do nosso cluster Kubernetes. Leia mais sobre isso aqui:

Services, Load Balancing, and Networking

GitHub - containernetworking/cni: Container Network Interface - networking for Linux containers

O Kubernetes dá os requisitos do que isso deve fazer, como por exemplo:

Pod para pod deve funcionar sem NAT

Qualquer Daemon no node precisa conseguir chegar nos Pods ali alocados

E então, as empresas criam as suas implementações. Ou seja, sempre que você cria um cluster Kubernetes, você precisa decidir qual CNI usar. Leia sobre as vantagens e desvantagens de cada uma delas antes de implementar.

Alguns exemplos:

<https://github.com/aws/amazon-vpc-cni-k8s>

<https://github.com/projectcalico/calico>

<https://github.com/cilium/cilium/tree/main>

A AWS VPC CNI é uma CNI desenvolvida para própria AWS fazendo a ponte com toda a infraestrutura da mesma, para rodar o EKS da forma mais otimizada possível. Esse é um ótimo exemplo do porque os providers que tem que desenvolver as CNIs.

O Calico é a opção “default” que a maioria utiliza em clusters On-premises.

O Cilium é a CNI que devo utilizar no meu homelab. A implementação dela é em cima do eBPF do Kernel Linux, que cria uma interface segura para interagir com coisas do Kernel, especialmente observabilidade.

Enquanto geralmente precisamos de um Service Mesh para fazer a observabilidade pod-to-pod, agora a própria CNI trás essa feature, o que é incrível!

Aqui tem a especificação da CNI:

<https://www.cni.dev/docs/spec/>

Como funciona a intercomunicação pela CNI? | Flannel

Conforme já falamos, cada CNI tem a sua forma de implementar o que precisa. Vamos falar especificamente da Flannel que é uma das CNI mais conhecidas e mais básicas também.

Primeiramente vamos listar as interfaces no nível do node:

```
$ ip a
```

Você vai notar algumas interfaces similares a essa aqui:

```
6: veth269ad138@if3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue  
master cni0 state UP group default
```

São as famosas Virtual Ethernet que vimos lá no treinamento de Fundamentos de Redes na parte de bridge. Veja que estão diretamente ligadas na interface cni0.

```
$ cd /var/run/netns
```

```
$ ls
```

```
$ nsenter --net=cni-<id>
```

Com isso eu já posso concluir que se um pod mandar requisição para outro pod do mesmo node, a comunicação será toda pela cni0.

A cni0 cobre um CIDR dentro do endereçamento maior de IPs da rede (e.g. 10.244.1.1/24). Ou seja, se eu enviar requisição para um pod de outro node, para onde isso vai?

Para saber isso, precisamos listar as rotas do node.

```
vagrant@worker-1:~$ ip r
```

```
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
```

```
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15
```

```
10.0.2.2 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
```

```
10.244.0.0/24 via 10.244.0.0 dev flannel.1 onlink
```

```
10.244.1.0/24 dev cni0 proto kernel scope link src 10.244.1.1
```

```
10.244.2.0/24 via 10.244.2.0 dev flannel.1 onlink
```

```
172.89.0.0/24 dev enp0s8 proto kernel scope link src 172.89.0.21
```

Como podemos ver, qualquer uma fora da subnet local, a requisição será enviada para a flannel.1. Existe um Daemon rodando no node, a CNI propriamente, que gerencia isso.

```
vagrant@worker-1:~$ ps aux | grep flannel
```

```
root    3457  0.0  1.7 1187712 35748 ?        Ssl  11:17   0:00 /opt/bin/flanneld --ip-masq --kube-subnet-mgr --iface=enp0s8
```

Quando chega na flannel.1, essa requisição será roteada pela interface enp0s8 que justamente conecta os nodes na camada L3, porém encapsulado usando VXLAN. Essa imagem pode ilustrar bem para você.

Query DNS dentro do cluster | CoreDNS

Primeiramente, é importante entender que os containers Linux funcionam da mesma forma que as VMs. As requisições de DNS serão enviadas para os servidores configurados em `/etc/resolv.conf`.

Ou seja, basta saber o que injetar nesse arquivo para fazer a resolução corretamente. Quem faz este trabalho é o próprio Kubelet do node onde o pod vai rodar, declarado no arquivo `/var/lib/kubelet/config.yaml` (pode mudar conforme o cluster).

clusterDNS:

- 10.233.0.3

Este será o endereço IP injetado nos pods.

Este é o Service balanceando a carga dos pods do CoreDNS.

```
$ kubectl get svc -A | grep 10.233.0.3
```

kube-system	coredns	ClusterIP	10.233.0.3	<none>
53/UDP,53/TCP,9153/TCP		167d		

Quem recebrá as requisições são esses dois pods aqui:

```
└─[$] <> kubectl get pods -n kube-system -l k8s-app=kube-dns
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-666959ff67-d728c	1/1	Running	0	14h
coredns-666959ff67-vlqvr	1/1	Running	0	14h

Para onde esses caras devem rotear a requisição depende do ConfigMap nessa mesma namespace, e também qual o DNS configurado no nível do Node.

Logs de debug CoreDNS

Para debugar o que está acontecendo na camada de DNS, podemos pegar os logs do CoreDNS. É claro que se houver algum problema de rede, a requisição não vai nem chegar no CoreDNS, então teria que abrir um pouco mais o leque de debugging e usar talvez um tcpdump da vida de dentro do pod fazendo a requisição.

Os logs do CoreDNS não vem habilitados por padrão, justamente porque vai gerar toneladas de logs. Então tenha em mente isso: Habilite apenas para debugging e desabilite quando finalizar.

```
$ kubectl edit cm -n kube-system coredns
```

O nome do ConfigMap pode mudar, ajuste para o seu caso.

E vamos basicamente usar isso aqui:

```
.:53 {  
    debug  
    log  
    ...  
    debug
```

```
$ kubectl rollout restart deploy -n kube-system coredns
```

ndots e seu impacto no DNS

Você vai notar que para cada query DNS, o CoreDNS fará sub-queries em alguns domínios.

Recomendo fortemente a leitura deste artigo complementar:

<https://pracucci.com/kubernetes-dns-resolution-ndots-options-and-why-it-may-affect-application-performances.html>

Com isso, podemos rodar uma query somente com o nome do Service, e se for na mesma namespace, ele vai completar o domínio e retornar o DNS correto.

Isso é puramente configuração do resolv.conf.

```
search kube-system.svc.cluster.local svc.cluster.local cluster.local default.svc.cluster.local  
mateusmuller.me  
nameserver 10.233.0.3  
options ndots:5
```

Em palavras simples... Todo domínio que tiver menos de 5 pontos separando as palavras, o CoreDNS vai injetar os local searches no final da URL.

Ou seja, uma query para teste.teste.teste.teste.

Vai ficar:

teste.teste.teste.teste.kube-system.svc.cluster.local

teste.teste.teste.teste.svc.cluster.local

...

Para a maioria dos casos isso não é performático. Imagina fazer isso em uma escala absurda de pods? Fica complicado.

Temos basicamente duas soluções:

Usar FQDN (adicionar um ponto “.” no final).

Sobrescrever o ndots por pod.

DNS externo pelo CoreDNS

Toda a configuração de resolução de nomes dos pods deve ser feita no CoreDNS, afinal todas as queries estão passando por ele.

Deixar eu te mostrar um caso de uso bem bacana.

Eu queria que toda a query destinada para o domínio *.mateusmuller.me fosse direcionada diretamente para o PiHole rodando dentro da minha infraestrutura. Isso para que os pods pudessem resolver os nomes da minha infraestrutura (jenkins, git, etc).

A configuração ficou mais ou menos assim:

```
mateusmuller.me {  
    log  
    errors  
    forward . 192.168.0.130  
    loadbalance  
    cache 30
```

```
    reload  
}
```

O importante aqui é:

```
forward . 192.168.100.12
```

Todas as requisições que envolva aquele domínio serão “proxeadas” para o upstream 192.168.100.12, que é justamente o PiHole.

Network Policies

Network Policies podem ser comparadas a um firewall na camada do Kubernetes, muito similar aos Security Groups da AWS também.

Com isso podemos delimitar qual pod tem acesso a qual pods.

Ao mesmo tempo que isso trás muita segurança, também trás um overhead muito grande para deixar tudo configurado, afinal nem sempre temos uma visão clara de quem deve falar com quem. Isso exige um bom conhecimento da arquitetura da aplicação, mas certamente é algo que leva sua estrutura de Kubernetes para o próximo nível.

Para testar a comunicação, vamos criar alguns manifestos demo e apenas mudar a imagem para o alpine.

O arquivo de network policies vou deixar em anexo nesta aula pois é um pouco grande.

Introdução ao RBAC

RBAC é um termo comum no mundo de infraestrutura, que serve para dar permissão de acesso à recursos privados baseado em roles (funções). Por exemplo, DevOps tem acesso a X, enquanto Desenvolvedores tem acesso a Y.

Isso vem muito de encontro com o princípio de least-privilege. Dar somente as permissões necessárias para quem realmente precisa, diminuindo os vetores de ataques.

No Kubernetes, temos um sistema de RBAC para controlar as chamadas do API server. Isso é especialmente útil em dois cenários:

Dar permissões para usuários finais que precisam de acesso ao nosso cluster Kubernetes (read-write? read-only?).

Dar permissões para Service Accounts sendo usadas por outros pods dentro do cluster.

CSR de usuário no Kubernetes

O primeiro passo é criar uma CSR (Certificate Signing Request) localmente. Essa CSR será enviada para a CA do Kubernetes assinar e devolver o real certificado.

```
openssl req -nodes \  
    -days 365 \  
    -newkey rsa:2048 \  
    -keyout estagiario.key \  
    -out estagiario.csr \  
    -subj '/CN=estagiario/O=mateusmullerme' \  
    -addext 'subjectAltName = DNS:estagiario.mateusmuller.me'
```

A assinatura do certificado será realizada dentro do próprio cluster via YAML. Para isso vamos encodar a CSR.

```
cat estagiario.csr | base64 -w 0
```

E usar o seguinte padrão:

```
apiVersion: certificates.k8s.io/v1
```

```
kind: CertificateSigningRequest
```

```
metadata:
```

```
  name: estagiario-csr
```

```
spec:
```

```
  request: <csr aqui>
```

```
  signerName: kubernetes.io/kube-apiserver-client
```

```
  usages:
```

```
    - client auth
```

Note que estamos pedindo um certificado do tipo client para autenticação com o cluster. Vou deixar algumas referências oficiais sobre isso aqui:

Authenticating

Certificates and Certificate Signing Requests

Manage TLS Certificates in a Cluster

Agora vamos aprovar a CSR, pegar o certificado e analisar o conteúdo.

```
kubectl certificate approve estagiario-csr
```

```
kubectl get csr estagiario-csr -ojson | \
    jq -r .status.certificate | \
    base64 -d > estagiario.crt
```

```
openssl x509 -in estagiario.crt -text
```

Note que foi assinado pela CA “CN = kubernetes”. Este certificado está pronto para ser usado no processo de autenticação.

Configurando usuário no kubectl context

Para configurarmos a autenticação do kubectl, precisamos editar o famoso kubeconfig. Ao invés de fazer isso “na munheca”, vamos usar o próprio kubectl que não tem erro.

```
$ kubectl config set-credentials estagiario \
    --client-certificate estagiario.crt \
    --client-key estagiario.key
```

```
$ kubectl config set-context estagiario \
    --cluster kind-kind \
    --user estagiario
```

```
$ kubectl config use-context estagiario
```

```
$ kubectl get pods
```

Provavelmente você vai receber um erro parecido com este aqui:

```
Error from server (Forbidden): pods is forbidden: User "estagiario" cannot list resource "pods"
in API group "" in the namespace "default"
```

Note que a autenticação já está funcionando. O Kubernetes identificou o nome do usuário baseado no CN do certificado e até mostrou na mensagem de erro!

A jogada agora é fazer o RBAC deste mesmo usuário.

ClusterRole e ClusterRoleBinding

Aqui você pode ver as RBACs que já vem em todos os clusters, default do Kubernetes mesmo.

Using RBAC Authorization

Existe uma ClusterRole muito interessante para os nossos estagiários: view. Pode ver as informações do cluster, mas não pode modificar (para evitar maiores problemas rsrs).

Bastaria a gente criar um binding mais ou menos assim:

```
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: ClusterRoleBinding
```

```
metadata:
```

```
  name: estagiario-ro
```

```
roleRef:
```

```
  apiGroup: rbac.authorization.k8s.io
```

```
  kind: ClusterRole
```

```
  name: view
```

```
subjects:
```

```
- kind: User
```

```
  name: estagiario
```

apiGroup: rbac.authorization.k8s.io

E problema resolvido! 😊

Role, RoleBinding e ServiceAccounts

Se você precisar liberar acesso aos recursos de namespace, use Role. Se o recurso for cluster-wide, use ClusterRole. Basicamente essa é a regra.

Podemos iniciar um pod com kubectl para testar isso.

apiVersion: v1

kind: Pod

metadata:

labels:

run: kubectl

name: kubectl

spec:

serviceAccountName: kubectl

containers:

- image: bitnami/kubectl

name: kubectl

command:

- sleep

- "9999999999999999"

Kubernetes API Concepts

Quando você executar alguma ação com kubectl, uma mensagem de erro será mostrada no terminal informando exatamente qual chamada você está fazendo e para qual objeto. Com isso, basta adicionar na Role para dar acesso.

Isso é interessante porque você apenas dá acesso aos recursos que a aplicação precisa e nada mais. O famoso princípio de least privilege.

Você sempre cria a Role e RoleBinding na namespace onde o recurso que você quer dar acesso está localizado. Já a ServiceAccount pode estar localizada em outra namespace.

A lógica das chamadas segue a mesma entre Role e ClusterRole.

A diferença é que a Role se aplica para recursos namespaced. Você pode filtrar todos com o seguinte comando:

```
$ kubectl api-resources --namespaced
```

kubent: APIs deprecadas

O primeiro passo na minha visão, além de planejar como será o upgrade:

console do cloud provider

kubespray (automatizado)

manual com kubeadm ou algo do gênero

blue-green? canary?

Isso tudo depende da criticidade, como o ambiente funciona, o que está rodando ali e outros fatores.

Além disso, precisamos identificar quais as APIs que foram deprecadas, e consequentemente serão removidas. Com isso, os manifestos aplicados, ou que forem aplicados, não funcionarão mais.

Aqui você consegue mais informações sobre isso:

Deprecated API Migration Guide

Mas claro que ainda assim podemos ter erros humanos, e é aqui onde entra o kubent.

GitHub - doitintl/kube-no-trouble: Easily check your clusters for use of deprecated APIs

Basicamente ele vai verificar os recursos aplicados no seu cluster, e cuspir na tela quais devem ser editados e porque, para que após o upgrade nada pare de funcionar.

Imagina que você está na versão 1.24 e quer ir para a versão 1.25.

```
$ kubent -t 1.25
```

```
>>> Deprecated APIs removed in 1.25 <<<
```

```
-----  
KIND    NAMESPACE NAME    API_VERSION  REPLACE_WITH (SINCE)  
CronJob default  hello   batch/v1beta1 batch/v1 (1.21.0)
```

Discussão sobre Upgrade

Existem várias formas de upgrade... Por exemplo no meu homelab, criado e gerenciado com kubespray, eu provavelmente faria algo seguindo esta documentação:

<https://github.com/kubernetes-sigs/kubespray/blob/master/docs/upgrades.md>

Porque é um processo homologado e automatizado. A dor de cabeça tende a ser bem menor.

Se o cluster estive rodando em algum cloud provider, como por exemplo a AWS, já tem um processo gerenciado pela AWS para isso.

Updating an Amazon EKS cluster Kubernetes version - Amazon EKS

Upgrade do Control Plane

Partindo do princípio que forma feitos todos os back-ups e medidas de disaster-recovery, vamos ao upgrade.

Precisamos configurar os pacotes da próxima versão. Então se o Kubernetes está na 1.24, vamos para a 1.25 (sempre pulamos apenas 1 minor version, nunca mais).

Installing kubeadm

Configure o repositório apt, atualize o cache e se prepare para instalar os pacotes da próxima versão.

```
curl -fsSL <https://pkgs.k8s.io/core:/stable:/v1.25/deb/Release.key> | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring-1.25.gpg
```

```
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] <https://pkgs.k8s.io/core:/stable:/v1.25/deb/> /' | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

```
apt update
```

```
apt list -a kubeadm
```

Com isso você deve ver todos os patches da nova minor. Os pacotes do Kubernetes são colocados on-hold para evitar upgrade ao atualizar o SO. Então vamos libera-los para o upgrade. A ideia é começar pelo kubeadm que controla as versões, e depois o kubectl e kubelet que são dependências.

```
apt-mark unhold kubeadm
```

```
apt install kubeadm
```

```
apt-mark hold kubeadm
```

```
kubeadm upgrade plan
```

Caso o plan tenha sucedido, basta prosseguir aplicando o upgrade efetivamente.

```
kubeadm upgrade apply v1.25.14
```

```
kubectl drain k8s-master-1 --ignore-daemonsets
```

```
apt-mark unhold kubectl kubelet
```

```
apt install kubectl kubelet -y
```

```
apt-mark hold kubectl kubelet
```

```
systemctl daemon-reload
```

```
systemctl restart kubelet
```

`kubectl uncordon k8s-master-1`

`kubectl get nodes`

Upgrade do Worker

Upgrading kubeadm clusters

O upgrade dos nodes é praticamente a mesma coisa, mas deve ser feito somente após o upgrade do control plane.

A diferença está após o upgrade do binário do kubeadm.

`kubeadm upgrade node`

Node Selector

Node Selector é a forma mais simples de scheduling, mas também a mais “limitada” de certa forma. A ideia é extremamente simples: Colocar o pod em nodes com uma label XYZ.

Isso é interessante para colocar workloads específicos em nodes específicos... CPU-bound? memory-bound? GPU (Machine Learning)?

Basta no nível do pod adicionar o seguinte:

nodeSelector:

kubernetes.io/hostname: k8s-worker-3

Qual o problema do nodeSelector? Ele é uma hard constraint. Se por falta de recursos ele não conseguir scheduler no node, o pod ficará em Pending, o que sinceramente não é o que queremos no dia a dia.

Seria melhor se tivéssemos algo como faça isso, mas se não der, não deixe de scheduler o pod.

Node Affinity

O Node Affinity é um Node Selector com muito mais robustez. Prefira sempre usar ele do que usar o Node Selector.

affinity:

nodeAffinity:

requiredDuringSchedulingIgnoredDuringExecution:

nodeSelectorTerms:

- matchExpressions:

- key: topology.kubernetes.io/zone

operator: In

values:

- antarctica-east1

- antarctica-west1

preferredDuringSchedulingIgnoredDuringExecution:

- weight: 1

preference:

matchExpressions:

- key: another-node-label-key

operator: In

values:

- another-node-label-value

O `requiredDuringSchedulingIgnoredDuringExecution` é basicamente a mesma coisa que o `Node Selector`, pois caso não seja acomodado, o pod ficará em `Pending`.

A diferença é o `preferredDuringSchedulingIgnoredDuringExecution`, onde o scheduler vai tentar seguir a sua constraint, e caso que não seja possível, ele ainda assim vai scheduler o pod.

Pod Affinity e Antiaffinity

Temos outro recurso bastante similar ao `NodeAffintity`, mas em nível de pod. Com isso podemos criar regras do tipo... “Coloque o pod X no mesmo node onde o pod Y está rodando”.

O mesmo segue para o inverso negando a regra citada acima.

Isso é interessante para casos onde a latência é algo crítico para a aplicação, então você pode colocar dois pods no mesmo node. Quando o pod A chamar o pod B, a latência será mínima porque estarão no mesmo node.

Por exemplo, colocar o pod de backend no mesmo node onde está a database para diminuir a latência de comunicação:

affinity:

podAffinity:

requiredDuringSchedulingIgnoredDuringExecution:

- labelSelector:

 - matchExpressions:

 - key: app

 - operator: In

 - values:

 - database

 - topologyKey: "kubernetes.io/hostname"

Isso não é um caso daqueles “one fits all”. Tudo deve ser analisado conforme os requisitos, prós e cons.

Pod Topology Spread Constraints

Este é um recurso muito interessante para controlar como os pods de um determinado workload são distribuídos dentro de uma(s) failure-domains, como por exemplo: nodes, zonas de disponibilidade, regiões, etc.

Isso pode garantir redução de custos, bem como uma maior disponibilidade da sua aplicação.

Pod Topology Spread Constraints

Então um exemplo de manifesto seria assim:

apiVersion: apps/v1

kind: Deployment

```
metadata:
  labels:
    app: nginx
  name: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: DoNotSchedule
          labelSelector:
            matchLabels:
              app: nginx
            matchLabelKeys:
              - pod-template-hash
      containers:
        - image: nginx
          name: nginx
```

maxSkew → A diferença de pods entre as constraints deve ser de 1.

topologyKey → Qual label vai ser utilizada para separar os pods.

whenUnsatisfiable → Caso a regra não seja atendida, o que fazer?

labelSelector → Qual label usar para atrelar pods do mesmo workload?

matchLabelKeys → Qual a label complementar a label assim (útil para o rolling update)

Para que seus pods não fiquem no status de Pending, utilize o whenUnsatisfiable: ScheduleAnyway. Nas palavras da própria documentação:

if you select whenUnsatisfiable: ScheduleAnyway, the scheduler gives higher precedence to topologies that would help reduce the skew.

Ou seja, o scheduler vai distribuir em diferentes Nodes mantendo a diferença de 1 pod (maxSkew 1).

Taints & Tolerations

Taint e Toleration é um recurso de scheduling extremamente elegante, e possivelmente o meu preferido. Com ele, podemos “proteger” os nodes de pods indesejados, onde somente quem foi explicito poderá ser schedulado lá.

Isso é uma estratégia interessante para segregar nodegroups (ex: frontend, backend, tools, etc). Podemos também ter ferramentas que necessitam de nodes dedicados, e essa estratégia também serve para isso.

Não é comum adicionar um taint manualmente em um node (exceto pelo drain para modo de manutenção). O mais comum é que o node já seja registrado com um taint específico, usando a flag --register-with-taints.

kubelet

Para fins de testes podemos adicionar um taint manualmente:

```
kubecttl taint node comunidade-devops-worker nodegroup=backend:NoSchedule
```

```
kubecttl create deploy --image nginx --replicas 10 nginx
```

Ao criar um deployment com vários pods, você vai notar que nenhum será schedulado no node com taint.

Para que isso aconteça, você precisa adicionar uma toleration explícita no manifesto. Lembrando que a toleration não é um selector, ele somente vai permitir que seja escalado nos nodes com taint.

apiVersion: apps/v1

kind: Deployment

metadata:

labels:

app: nginx

name: nginx

spec:

replicas: 10

selector:

matchLabels:

app: nginx

template:

metadata:

labels:

app: nginx

spec:

tolerations:

- key: nodegroup

operator: Equal

value: backend

effect: NoSchedule

containers:

- image: nginx

name: nginx

ImagePullBackOff & CrashLoopBackOff

Diariamente na sua experiência com Kubernetes, você verá dois erros enquanto os pods ficam crashando: ImagePullBackOff & CrashLoopBackOff.

Vamos falar um pouco sobre cada um deles, e quais comandos utilizar para analisar o problema.

ImagePullBackOff

Durante o startup do pod, o node ao qual está assignado vai fazer o pull da imagem de um registry. Esse processo pode falhar por vários motivos:

autenticação (mais comum)

tag inexistente

rede (timeout / latencia)

tls (ex certificado expirado)

No fim das contas o erro no pod vai ser o mesmo: ImagePullBackOff. Nesses casos, o comando que precisar ser usado é o `kubectl describe`. Nesse momento o pod nem startou ainda, então logs ou exec não são úteis.

Vamos simular o problema e executar o describe. Vou colocar uma tag inexistente.

apiVersion: apps/v1

kind: Deployment

metadata:

labels:

app: alpine

name: alpine

spec:

replicas: 1

```
selector:
  matchLabels:
    app: alpine
template:
  metadata:
    labels:
      app: alpine
  spec:
    containers:
      - image: alpine:vraus
        name: alpine
        command:
          - sleep
          - infinity
```

E vamos dar describe no pod falhando.

```
$ kubectl describe pod <pod>
```

Você verá os eventos gravados para o startup do pod, e um deles será a causa do BackOff.

```
Warning Failed 21s (x2 over 42s) kubelet Failed to pull image "alpine:vraus": rpc
error: code = NotFound desc = failed to pull and unpack image
"docker.io/library/alpine:vraus": failed to resolve reference "docker.io/library/alpine:vraus":
docker.io/library/alpine:vraus: not found
```

Claramente o erro é: Essa imagem não existe.

CrashLoopBackOff

O CrashLoopBackOff é o erro mais comum eu diria, e é essencialmente erro na aplicação. Os outros times vão vir quebrando falando que é problema no Kubernetes e bláblá, quando na verdade o processo principal do container está finalizando com código de erro diferente de zero (erro).

Então aqui os comandos invertem: O describe não vai trazer muitos detalhes, porque o pod já startou, e o erro é na aplicação dentro do pod. O log é o subcomando mais efetivo para estes casos.

Vamos simular aqui, sobrescrevendo o CMD da imagem para um ls -l em uma pasta que não existe. O ls -l vai retornar um código diferente de zero e vai crashar o pod.

apiVersion: apps/v1

kind: Deployment

metadata:

labels:

app: alpine

name: alpine

spec:

replicas: 1

selector:

matchLabels:

app: alpine

template:

metadata:

labels:

app: alpine

spec:

containers:

- image: alpine:latest

name: alpine

command:

- ls

- -l

- vraus

Então vamos usar o logs:

```
└─[$] <> kubectl logs alpine-6894474c59-6kfqz
```

ls: vtaus: No such file or directory

Isso explicaria o porque o pod está crashando. O mesmo serve para qualquer outra aplicação, nos logs você entenderá olhando a exceção de erro do porque está crashando.

Simulando NodeNotReady

O kubelet é o agente que rodar em cada node, e é responsável pelos pods que lá estão. Isso é, realizar os healthchecks dos probes, fazer pull de imagem, etc.

Além do gerenciamento normal, ele é o cara que fala diretamente com o API server para o node entrar no cluster. Para lembrar, veja a imagem dessa doc:

<https://kubernetes.io/docs/concepts/overview/components/>

Então, para que toda essa comunicação entre os worker nodes e os master nodes ocorra da melhor forma possível, a rede e segurança precisa estar propriamente configurada.

Aqui dá para ter uma ideia de todas as portas que precisam estar abertas:

<https://kubernetes.io/docs/reference/networking/ports-and-protocols/>

Em Cloud seria as route tables, security groups, NACLs, etc.

Em um ambiente on-premises por exemplo, o IPTables pode ser um impeditivo se algo foi configurado erroneamente.

Vamos simular um problema, para que você entenda quando e onde conseguir esse log.

Vou acessar via SSH os 3 master nodes do meu cluster Kubernetes, e dentro de cada um deles, eu vou dropar todo o tráfego vindo de um worker específico.

```
$ iptables-save > /tmp/iptables.bkp # back-up por garantia
```

```
$ iptables -I INPUT -s 192.168.0.41 -j DROP
```

No caso acima, usei o -I INPUT para jogar a regra no topo da cadeia, acima até mesmo de regras de CNI e kube-proxy.

Em alguns minutos, o worker node vai mudar o status de Ready para NotReady.

NAME	STATUS	ROLES	AGE	VERSION
k8s-master-1	Ready	control-plane	273d	v1.28.2
k8s-master-2	Ready	control-plane	273d	v1.28.2
k8s-master-3	Ready	control-plane	273d	v1.28.2
k8s-worker-1	Ready	<none>	273d	v1.28.2
k8s-worker-2	NotReady	<none>	273d	v1.28.2 -----> aqui
k8s-worker-3	Ready	<none>	273d	v1.28.2
k8s-worker-4	Ready	<none>	273d	v1.28.2

Sempre que você se deparar com uma situação assim, ao invés de cegamente começar a testar um caminho de coisas, acesse o worker node via SSH ou alguma outra forma e colete o log do Kubelet.

```
$ journalctl -fu kubelet
```

```
Jun 26 11:54:29 k8s-worker-2 kubelet[1333]: E0626 11:54:29.538499 1333 controller.go:146]
"Failed to ensure lease exists, will retry" err="Get
\\<https://localhost:6443/apis/coordination.k8s.io/v1/namespaces/kube-node-
lease/leases/k8s-worker-2?timeout=10s\\>": context deadline exceeded - error from a
previous attempt: EOF" interval="7s"
```

```
Jun 26 11:54:32 k8s-worker-2 kubelet[1333]: E0626 11:54:32.438807 1333
kubelet_node_status.go:540] "Error updating node status, will retry" err="error getting node
\\<k8s-worker-2\\>: Get \\<https://localhost:6443/api/v1/nodes/k8s-worker-
2?resourceVersion=0&timeout=10s\\>": context deadline exceeded - error from a previous
attempt: EOF"
```

Isso é, o node não consegue até chegar o API server na porta 6443. Nesse momento é onde você começa a usar ferramentas como netcat/curl para testar a comunicação. Eventualmente você vai chegar na solução, que nesse caso é regra de firewall.

PS: para voltar ao normal usei:

```
$ iptables -D INPUT 1
```

Estratégias de teste (sidecar & pod)

Diariamente precisamos testar a comunicação pod-to-pod, passando por service e todos os recursos do ecossistema do Kubernetes.

Como a comunicação é totalmente interna, precisamos de um container de teste com ferramentas de rede para validar a comunicação.

Pod de teste

Para esses casos eu costumo iniciar um pod de teste com alpine e instalar o que preciso. Nem sempre isso é possível, mas é comum o time ter uma imagem customizada com ferramentas de teste.

Assim eu rapidamente gero um manifesto para um pod do alpine:

```
$ kubectl run --image alpine --dry-run -oyaml alpine | kubectl neat > alpine.yaml
```

Então eu só adiciono um command com sleep para travar o pod e eu pode acessar.

```
$ vim alpine.yaml
```

command:

- sleep
- infinity

```
$ kubectl apply -f alpine
```

```
$ kubectl exec -it alpine sh
```

```
# apk add curl
```

E aí é só correr para o abraço nos testes.

Sidecar container

Porém, nem sempre isso é possível, ou a melhor opção. Há vezes que precisamos inspecionar a network namespace do pod, mas não temos as ferramentas necessárias para isso, então se faz necessário injetar um sidecar para isso.

Imagina que temos um deployment com NGINX rodando.

```
$ kubectl create deploy --image nginx nginx
```

Podemos alterar o deployment, incluindo um container sidecar do alpine também.

```
$ kubectl edit deploy nginx
```

spec:

containers:

- image: alpine

- name: alpine

- command:

- sleep

- infinity

Quando o novo pod for iniciando, basta darmos o mesmo exec informando o container.

```
$ kubectl exec -it nginx-5784699764-8qnbh -c alpine sh
```

```
/ # apk add curl
```

```
fetch <https://dl-cdn.alpinelinux.org/alpine/v3.20/main/x86_64/APKINDEX.tar.gz>
```

```
fetch <https://dl-cdn.alpinelinux.org/alpine/v3.20/community/x86_64/APKINDEX.t>
```

```
...
```

```
...
```

```
OK: 13 MiB in 24 packages
```

```
/ # curl localhost
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Welcome to nginx!</title>
```

```
...
```

...

Canivete suíço!

E também há casos onde você não tem nem permissão para instalar pacotes, então a imagem do alpine não vai ajudar muito.

Se você não tem nenhuma imagem pronta para troubleshooting, recomendo a netshoot como um bom ponto para você iniciar.

<https://github.com/nicolaka/netshoot>

```
$ docker run -it nicolaka/netshoot sh
```

Eu duvido que nenhuma das ferramentas incluídas não te ajude. A imagem já vem com tcpdump, netstat, iperf, etc.

```
k auth can-i create pod --as system:serviceaccount:default:dev-sa  
--as system:resource:namespace:resourceNAME
```

k = kubectl

auth:

Available Commands:

can-i	Check whether an action is allowed
reconcile	Reconciles rules for RBAC role, role binding, cluster role, and cluster role binding objects
whoami	Experimental: Check self subject attributes

ISSO AQUI É OURO!!!!

```
controlplane $ k edit svc kube-dns -n kube-system
```

```
error: services "kube-dns" is invalid
```

```
A copy of your changes has been stored to "/tmp/kubectl-edit-3488213665.yaml"
```

```
error: Edit cancelled, no valid changes were saved.
```

```
controlplane $ kubectl replace -f /tmp/kubectl-edit-3488213665.yaml --force
```

```
service "kube-dns" deleted
```

```
service/kube-dns replaced
```

kustomize patches

op - add/remove/replace

target

Kind

Version/Group

Name

Namespace

labelSelector

AnnotationSelector

Value: Valor que sera substituido ou adicionado (nao aplicavel para recursos que sarao removidos)

json 6902

replace operation -----

patches:

- target:

kind: Deployment

name: api-deployment

patch: |-

- op: replace

path: /spec/replicas

value: 5

Em lista:

patches:

- target:

kind: Deployment

name: api-deployment

patch: |-

- op: replace

path: /spec/template/spec/containers/0 # -> O zero é o primeiro item em
uma lista

value:

name: haproxy

image: haproxy

add operation -----

patches:

- target:

kind: Deployment

name: api-deployment

patch: |-

- op: add

path: /spec/template/metadata/labels/newLabel

value: novaEtiqueta

Em lista:

patch: |-

- op: add

path: /spec/template/spec/containers/- # -> O dash é o ultimo item em uma lista

value:

name: nginx

image: nginx

remove operation -----

patches:

- target:

kind: Deployment

name: api-deployment

patch: |-

- op: remove

path: /spec/template/metadata/labels/newLabel

Em lista:

patch: |-

- op: remove

path: /spec/template/spec/containers/1 # -> O um é o segundo item em uma lista (aqui é necessário especificar a posição do recurso que voce deseja remover)

Strategic Merge Patch (Mantem a estrutura original do arquivo focado apenas no recurso que sera aplicado o patch)

patches:

- patch: |-

apiVersion: apps/v1

kind: Deployment

```
metadata:
  name: api-deployment
spec:
  replicas: 5
```

Nesse metodo, quando for remover um recurso (label, por exemplo), o valor deve ser especificado como "null".

patches:

```
- patch: |-
  apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: api-deployment
  spec:
    template:
      metadata:
        labels:
          newLabel: null
```

E no caso de uma lista, é necessário especificar a "delete directive"

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  containers:
    - name: web
      image: nginx
    - name: database      # -> voce deseja deletar esse recurso
      image: mongo
```

Como ficaria:

- patch: |-

apiVersion: apps/v1

kind: Deployment

metadata:

name: api-deployment

spec:

containers:

- \$patch: delete # -> "delete directive"

name: database